

Infosys Technologies Limited

Education and Research Department

Unix

November, 2001

Document No.	Authorized By	Ver.Revision	Signature / Date
ER/CORP/CRS/OS04/002	Dr.Ravindra M. P.	3.0	

COMPANY CONFIDENTIAL

Modification Log

Ver. Revision	Date	Author(s)	Description
2.00	03/05/94	Mohan Kumar C	Original Version
2.01	24/08/96	Benny Raphael	Introduction to system calls has been added as Appendix-B
2.1	21/07/97	Reena R	Appendix-B has been changed to Chapter-10. Some topics are explained in more detail and exercises have been modified.
2.11	28/05/98	Reena R	Based on the last few feedbacks on this course, we have incorporated following changes/additions 1. Commands: For instance kill, sort and expr with examples have been included. 2. AWK: Examples have been included to illustrate the flow-control statements. 3. VI: Exercises for mapping features are included. Apart from this, corrected the errors, connected the flow-of-control of topics and unified the formatting structure across chapters.
2.12	11/11/98	Reena R.	Separated Course Description & Course Design
3.0	15/11/2001	Eugene Xavier.P.	Deleted chapters on AWK and System Calls – Included new commands in Chapter 2 – Included Guidelines to write Shell scripts in chapter 3 . Modified Chapter 6 on Unix Communications

Review Record

Version. Revision	Date	Reviewer(s)	Description	Remarks by the Author
3.0	15/11/2001	Savita.C Ramesh Babu.S. Kamalakannan.L. Vinay K.M.	The reviewers' comments: Code snippets have to be included to bring practices into class room. Chapters on AWK and System calls have to be removed from the course material. Chapter on Unix communication has to be modified. Some more important Commands have to be included. Introduction to IPC has to be included.	Incorporated all the necessary modifications to be made as suggested by Reviewers.

UNIX Course Description

Course number	OS04	Course name	Unix
Author	Eugene Xavier. P		(Introductory Concepts)
Pre-requisites :	Elementary understanding of any operating system. CHSSC Course		
Estimated effort for preparation	About 20 days		
Estimated course duration	5 days (10.5 hours of lecture, Rest of the sessions are Tutorials/Hands-on/Assignments/Project)		

Course objectives (course objectives state the major outcomes of the training and indicate how to measure the expected performance)		
Sl#	Objectives (Each one to be stated as one or more complete sentences)	Demonstrable knowledge/skills
1.	To acquaint participants to the UNIX operating system	Ability to understand an overview and major components in Unix operating system such as File systems, Kernel, processing environment
2.	To develop an ability to use general Unix commands	Ability to use simple Unix commands like, ls, cp, mv, ln, rm, cat, chown, ps, who
3.	To develop ability to write programs in shell.	Ability to understand and program in shell using shell variables, loops and signals
4.	To develop ability in using Unix utilities	Ability to understand and use utilities like vi editor, make, SCCS
5..	To acquaint participants the security issues in Unix	Ability to appreciate the need for security and put them in practice, like using file permissions, passwords, setuid and file encryption.

Sources consulted :

1. *The UNIX Programming Environment* – Brian W. Kernighan and Rob Pike
2. *Advanced UNIX - A Programmer's Guide* – Stephen Prata
3. *Exploring the Unix System* – Stephen G. Kochan and Patrick H. Wood
4. *Understanding Unix - A conceptual guide* – James R. Groff, Paul N. Weinberg
5. *The Design of the Unix Operating System* – Maurice J. Bach
6. Personal Communication with PLs.

Session Plan

Units					
Day	Sl#	Unit name	Unit objectives and keywords	Coverage strategy	Est time (deliver)
Day 1	1	Introduction to Unix Operating System	Unix Operating system architecture, Filesystem, Processing environment, Operating system services.	Lecture	1.0 hours
	2	General Unix commands	Unix commands: ls, cd, cat, who, ps	Lecture	1.5 hours
				Hands-on	6.0 hours
Day 2	3	vi Editor	Vi editor command for cursor movements, insert mode, command mode and colon mode commands, Set command and handling multiple files	Lecture	1.0 Hours
				Hands-on	4.0 hours
	4	Text processing utilities	Commands like wc, head, tail, cut, paste, redirection, pipes and pattern searching.	Lecture	1.0 hour
				Hands-on	1.5 hours
	5	Advanced commands	Commands like find, file, touch, du and df, ulimit and umask Trainees should read the chapter on Shell programming in the course material.	Lecture	0.5 hours
				Hands-on	0.5 hours
Day 3	6	Shell programming – I	Shell variables, export, read and echo commands	Lecture	1.0 hours
				Hands-on	2.0 hours
ER/CORP/CRS/OS04/002					
Copy if printed					

	7	Shell programming – II	Commands like while, until, set, for, case.. esac, examples Project Specification and Test Plan are distributed. Project will be done in groups of two. Trainees should be asked to prepare test plan first and then only our test plan should be given	Lecture Hands-on	1.5 hours 4.0 hours
Day 4	8	UNIX Shells & Korn shell	Sh, Csh, Korn and bash, nohup and nice commands	Lecture Hands-on	1.0 hours 1.0 hours
	9	Advanced shell programming	.profile, . command, trap, eval and tput command	Lecture Hands-on	0.5 hours 0.5 hours
	10	Utilities	Commands for compress: gzip, tar etc Communication: write, wall, ftp, telnet, Introduction to IPC Discussion about project (clarification of doubts)	Lecture Hands-on & project	1.5 hours 4.5 hours
		Project	Project Demo based on the Test Plan Inter group evaluation		2.0 hours

Table of contents

1	UNIX - GENERAL OVERVIEW	1
1.1	THE UNIX OPERATING SYSTEM	1
1.2	UNIX SYSTEM ARCHITECTURE	1
1.3	THE USER PERSPECTIVE	4
1.4	OPERATING SYSTEM SERVICES.....	5
2	GENERAL UNIX COMMANDS	9
2.1	SOME UNIX COMMANDS.....	9
2.2	SOME OTHER UTILITIES.....	15
2.3	REGULAR EXPRESSIONS.....	23
3	FUNDAMENTALS OF UNIX SHELL PROGRAMMING.....	27
3.1	SHELL FUNCTIONS.....	27
3.2	SHELL VARIABLES	27
3.3	THE SHELL - METACHARACTERS AND SPECIAL SYMBOLS.....	30
3.4	LOOPING AND MAKING CHOICES	32
3.5	THE TEST COMMAND	38
3.6	THE EXPR COMMAND.....	40
3.7	ERROR CHECKING IN SHELL PROGRAMS	41
3.8	THE EXPORT COMMAND.....	44
3.9	ENVIRONMENT SETTING.....	47
3.10	FULL BACKUP SCRIPT	52
3.11	BACKGROUND PROCESSING.....	54
3.12	GUIDELINES FOR UNIX SHELL SCRIPTS.....	57
4	THE VI EDITOR.....	62
4.1	VI : CAUTIONS	62
4.2	VI : SALIENT FEATURES.....	62
4.3	VI : THE BASICS.....	62
4.4	VI : IMPORTANT KEYS	63
4.5	VI : INVOKING	63
4.6	VI : CURSOR MOVEMENTS (CMD MODE COMMANDS)	63
4.7	VI : SCREEN COMMANDS -SCROLLING (CMD MODE COMMANDS).....	64
4.8	VI : DELETING DATA (CMD MODE COMMANDS)	65
4.9	VI : TEXT INSERTION	65
4.10	VI : REPLACING EXISTING TEXT	65
4.11	VI : CUT AND PASTE.....	66
4.12	VI : SAVING AND EXITING (COLON : COMMANDS).....	66
4.13	VI : BLOCK MOVES (COLON : COMMANDS).....	66
4.14	VI : 9 LIVES.....	67
4.15	VI : USER DEFINED REGISTERS.....	67
4.16	VI : MARKING AND YANKING	67
4.17	VI : PATTERN SEARCHING	68
4.18	VI : SUBSTITUTE COMMANDS(COLON : COMMANDS).....	68
4.19	VI : MACROS.....	68
4.20	VI : MAPPING KEYS TO COMMANDS.....	68
4.21	VI : RUNNING SHELL COMMANDS.....	69
4.22	VI : READING COMMAND OUTPUT	69
4.23	VI : EDITING MULTIPLE FILES SIMULTANEOUSLY	69
4.24	VI : READ AND WRITE ACROSS FILES.....	70
4.25	VI : THE SET OPTIONS.....	70

4.26	VI: THE .EXRC FILE	71
5	UNIX - SECURITY FEATURES	75
5.1	PASSWORD - THE SHIELD	75
5.2	GOOD PASSWORDS.....	75
5.3	CHANGING PASSWORDS	76
5.4	FILE SECURITY.....	76
5.5	DIRECTORY PERMISSIONS IN UNIX	78
5.6	THE STICKY BIT - ULTIMATE DIRECTORY PROTECTION.....	79
5.7	THE SU COMMAND	79
5.8	THE SU-ID BIT - A TEMPORARY PRIVILEGE	80
5.9	THE RESTRICTED SHELL	80
5.10	FILE ENCRYPTION.....	80
6	ELEMENTARY UNIX - UNIX COMMUNICATIONS.....	82
6.1	TELEPHONE COMMUNICATIONS	82
6.2	MODERN COMMUNICATION METHODS.....	82
7	UNIX - PROGRAM DEVELOPMENT.....	86
7.1	C PROGRAM DEVELOPMENT.....	86
7.2	C PROGRAMMING TOOLS	87
8	AN INTRODUCTION TO INTERPROCESS COMMUNICATION	96

1 UNIX - GENERAL OVERVIEW

Keywords in this chapter

<i>Time Sharing</i>	<i>Kernel</i>	<i>Portability</i>	<i>System Calls</i>
<i>Shell</i>	<i>Interpreter</i>	<i>Process</i>	<i>Primitives</i>
<i>Pipes</i>	<i>Interrupts</i>	<i>Exceptions</i>	<i>Re-direction</i>

1.1 The Unix operating system

Unix is a time sharing operating system (OS), which consist of kernel, filesystem, shell and a collection of tools and utilities.

The characteristic features of the Unix OS:

1. Written in C - easy understanding and portability of code i.e., can run on micros to mainframes
2. Interactive - has a consistently good user interface
3. Based on the philosophy of primitives to build complex programs
4. Easy to implement and maintain - hierarchical file system incorporating de-mountable volumes -
5. Easy application development - consistent format for byte streams/files
6. Simple and consistent interface to peripheral devices (treated as files) - compatible file, device and inter-process I/O
7. Supports multi-user and multi-processor modes
8. Easier to write portable programs - hides machine architecture from the user -
9. Support any language that has an interpreter or compiler
10. Can initiate asynchronous processes

In general, like any OS, Unix provides the following two classes of services:

1. User services - shell, mail, text-processing, source code control system (SCCS)etc.
2. System calls - Operating system services for application development

1.2 Unix system architecture

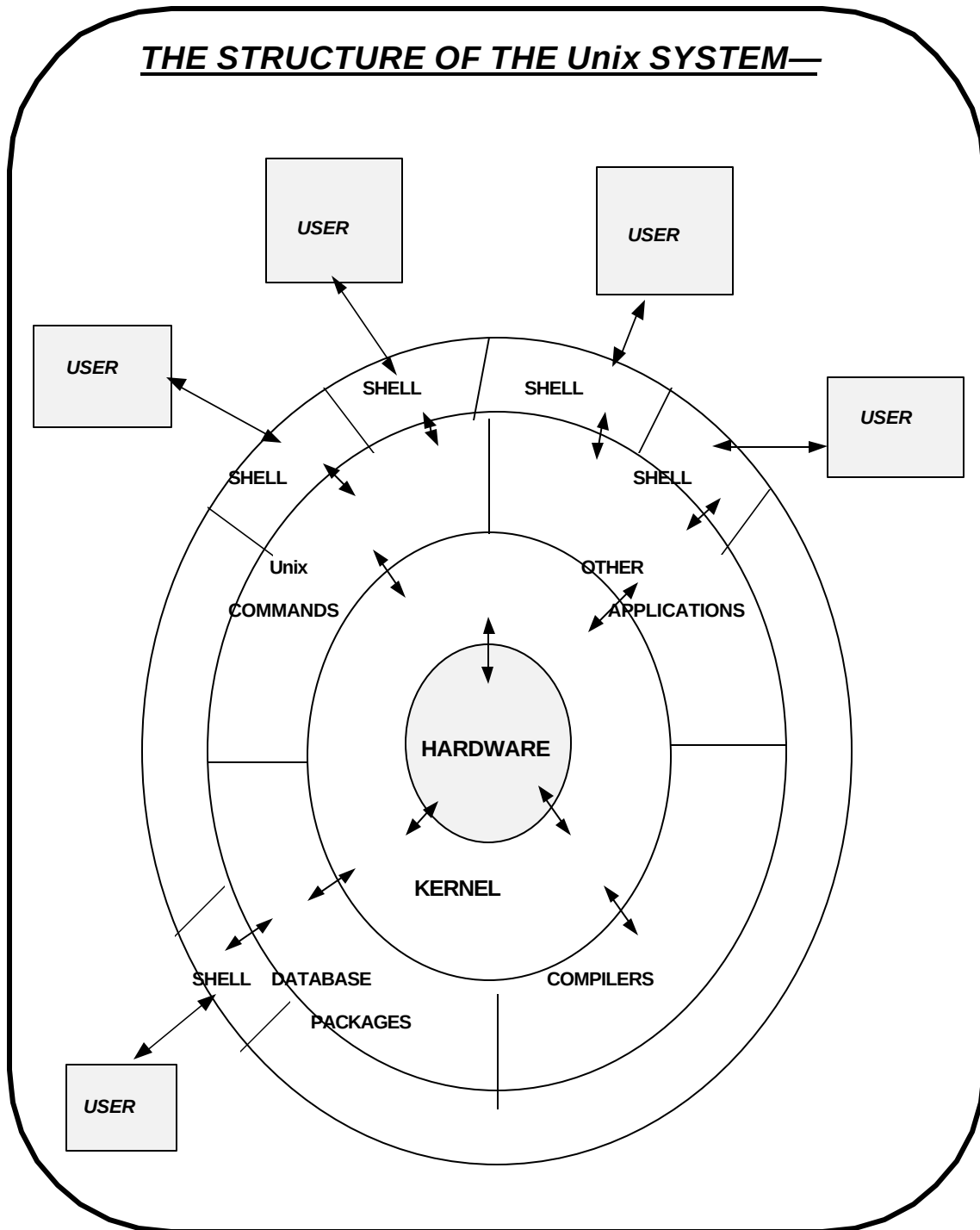
Unix follows a layered approach that facilitates easy interaction between the different modules of the OS

Layer 1: This layer is the hardware over which the OS layers are built. This layer is not a part of Unix OS

Layer 2: This layer is the kernel part of OS which interacts directly with the hardware providing common services to programs and insulating them from hardware idiosyncrasies. It is a program that monitor and controls the resources of a computer and allocates them among its users in an optimal manner.

Layer 3: This layer contains programs that interact with the kernel by invoking a well-defined set of system calls. Typically, these programs are Unix commands like ex, wc, grep etc., and may include certain user-developed programs also.

Layer 4: This layer contains other application programs that have built on the lower levels like SCCS



1.3 The user perspective

From the users viewpoint the following aspects of Unix will be highlighted:

1. File system
2. Processing environment
3. Building block primitives

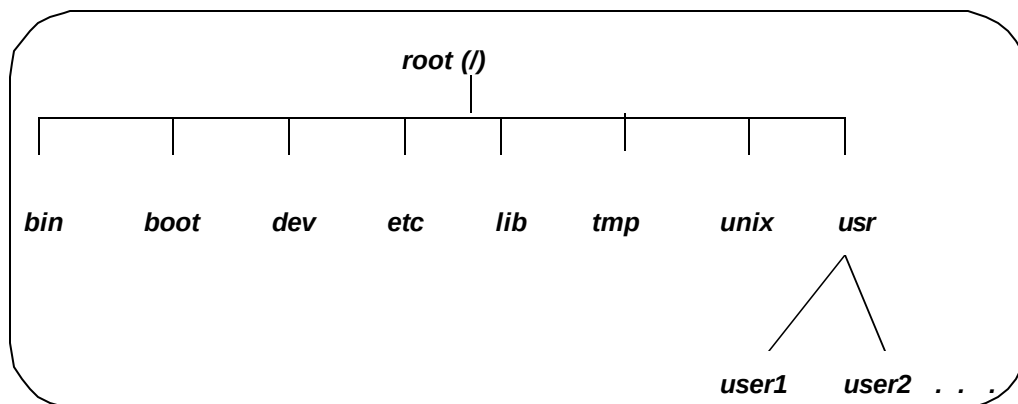
1.3.1 The file system

The main features of a Unix file system:

1. hierarchical structure
2. consistent treatment of file data
3. create and delete files
4. dynamic growth of files
5. protection of file data
6. peripheral devices as files

The file system follows a tree structure with root represented by '/'. Every non-leaf node is a directory of files and the leaf nodes can be directories, regular files or special device files.

Typical structure of the Unix file system



File access permissions can be set 3 classes of users:

1. File Owner
2. File Owner's Group
3. Others

Read (r), Write (w), Execute (x) permissions can be set for each file.

1.3.2 The processing environment

A program is just an executable file whereas a process is an instance of a program in execution. The creation /

termination of processes, the synchronization and reaction to interrupts is controlled through system calls. An example of a program using the kernel through system calls is the shell - the COMMAND INTERPRETER. The shell is a user program that can be tailored to the needs of the user. The shell can also execute commands asynchronously i.e. without waiting for the prior command to terminate, background execution. The shell provides a processing environment for the user programs.

The shell offers the following facilities:

1. File name short-hands through metacharacters
2. Input/Output (I/O) redirection
3. Personalizing the environment
4. Defining new commands and short-hand macros

The most powerful feature of the shell is, it is programmable. Shell scripts, which are a series of shell commands, can be executed using the shell.

1.3.3 Building block primitives

The Unix philosophy supports building complex programs from simple primitives. The following are supported

1.3.3.1 Redirection of I/O

Both the standard input and the standard output can be re-directed from/into files:

\$ ls > output

Here the output from the ls command gets written into the file output.

\$ mail raj < letter

The content of the file *letter* is mailed to user raj.

\$ nroff -mm < doc1 > doc1.out 2 > errors

Standard input/output and error re-directed simultaneously.

1.3.3.2 Pipes

A pipe is a mechanism that allows a stream of data to be passed between the read and the write processes.

\$ grep they x.c y.c z.c | wc -l

Counts the number of lines in files x.c, y.c and z.c that contain the word 'they'; the output from grep is 'piped' directly as input to wc.

Note: In most situation pipes makes it unnecessary to create temporary files.

1.4 Operating System Services

1.4.1 Kernel

The Kernel performs various primitive operations on behalf of user programs:

1. Control of execution of processes - creation, synchronization, termination, suspension and communication of processes.
2. Scheduling of processes - deciding of time-sharing of processes.

3. Memory management - Allocation of memory for executing processes - two methods are used:

Swapping: Entire process written to swap device to accommodate another process.

Paging: Pages of memory written to swap device.

These overcome the limitations of simple time-sharing.

4. File system service - Allocation of secondary storage efficiently for user data files.
5. Controlled access to peripherals, which are treated as special files.

1.4.2 Hardware transparency

Processes usually run in one of the two modes:

1. *User mode* - cannot access kernel instructions and data
2. *Kernel mode* - can access kernel and user addresses and privileged instructions not allowed for user mode.

The hardware only view processes in terms of kernel or user modes. It does not distinguish between the user processes and the OS has internal bookkeeping to distinguish between processes.

1.4.3 Interrupts and exceptions

Interrupt: Interrupt is a signal by which priorities and instruction-scheduling sequence can be controlled. The kernel saves the current *context* (a frozen image of what the processes was doing) when an interrupt occurs. After servicing the interrupt the context is restored and the process proceeds. The order in which an interrupt is serviced depends upon its priority.

Exception: An exception is an unexpected event like reference to an illegitimate address caused by events external to a process - exceptions usually occur in the middle of a process where as interrupts occur between them. Different algorithms are used to handle interrupts/exceptions efficiently.

1.4.4 Processor execution levels

Based on the priority level of an interrupt the kernel masks certain interrupts in order that a critical process may proceed without error. The priority levels in descending order are:

1. Machine error
2. Clock
3. Disk
4. Network Devices
5. Terminals
6. Software Interrupts

An example would be that there be no disk interrupts while manipulating linked lists.

1.4.5 Memory management

The concept of a *Virtual Machine* is central to memory management under Unix. The compiler generates addresses for a *virtual machine* as if no other program will execute simultaneously on the physical machine. Virtual addresses generated by the compiler need not be identical to the actual physical addresses in the machine. The memory management coordinates with the machine hardware to set-up a virtual to physical address translation that maps the compiler-generated address to the physical machine address.

Assignment - 1

1. *What does a process mean?*
2. *What is the advantage of each user having his own copy of the shell?*
3. *What do you understand by 'building block primitive'? Give examples.*
4. *Distinguish between Paging and Swapping.*
5. *Distinguish between User mode and Kernel mode.*
6. *Distinguish between Interrupts and Exceptions. Give examples.*
7. *What is a Virtual Machine?*
8. *Distinguish between physical addresses and logical addresses.*
9. *Distinguish between Multi-tasking, Multi-user, Multi-processing and Time-sharing.*
10. *What are processor execution levels and priorities?*
11. *What do you understand by 'Unix is a Portable OS'?*
12. *What do you understand by the term 'de-mountable volumes'?*

2 GENERAL UNIX COMMANDS

Keywords in this chapter

<i>ls</i>	<i>cp</i>	<i>Mv</i>	<i>ps</i>
<i>rm</i>	<i>cat</i>	<i>Pwd</i>	<i>cd</i>
<i>mkdir</i>	<i>Rmdir</i>	<i>Chmod</i>	<i>chown</i>
<i>chgrp</i>	<i>Who</i>	<i>Head</i>	<i>tail</i>
<i>uniq</i>	<i>Tr</i>	<i>Wc</i>	<i>grep</i>

2.1 Some Unix commands

2.1.1 ls - list files in a directory

\$ ls [option] [filenames]

Output is sorted alphabetically by default.

- Option -a list includes files starting with (.)
- Option -i lists the i-node numbers.
- Option -l lists in long formats - includes the mode, number of links, owner, group, size in bytes, time of last modification and name of the file.
- Option -r lists in the reverse order.

Example:

```

$ ls          $ ls -i
prog1        44      prog1
prog2        56      prog2
$
$ ls -a
.
..
.profile
prog1
prog2

```

\$

\$ ls -l prog1 -rwxr-xr-- 1 john project 120 oct 12 1345 prog1

2.1.2 cp - copy files

\$ cp file1 file2

'file1' is copied to 'file2'

\$ cp file1 file2 directory1

Files are copied into the 'directory1'. Directory must exist before the execution of the command.

Example:

\$ cp test example

\$ cp test.c test1.c test2.c project

2.1.3 mv - move or rename files or directories

\$ mv [-f] file1 file2

\$ mv [-f] directory1 directory2

\$ mv [-f] file ... directory

If file2 already exists, it is removed before file1 is moved.

If file2 has a mode forbidding writing, the mode is printed and reads the standard input to find whether the move should take place or not.

Example:

\$ mv test example1

\$ mv project sample

\$ mv test.c test1.c test2.c testdir

2.1.4 ln - make a link to a file

\$ ln file1 [file2 ...] target

A link is a directory entry to a file.

Any changes to a file are effective, independent of the name by which the file is known.

Example:

\$ ln prog1 prog1.bat

This is an example of hard link. Prog1 should already exist and prog1.bat should not exist already. If prog1 is removed

prog1.bat is not deleted. Prog1 and prog1.bat have same inode numbers (that could be seen in the long listing)

```
$ ln -s prog1 /tmp/prog1.bat
```

This is an example of symbolic link. prog1 should already exist and prog1.bat in tmp directory should not exist. This creates a link to prog1.bat under tmp directory. If prog1 is removed prog1.bat is deleted. Prog1 and prog1.bat have different inode numbers (that could be seen in the long listing). → could be seen in the long list of the symbolically linked file entries.

2.1.5 rm - remove files or directories

```
$ rm [option...] file ...
```

Removes one or more files from a directory.

Removal of a file requires write permission in its directory.

- Option -f force the removal of files that do not have write permission.
- Option -r recursively delete the entire contents of directories specified and the directories themselves.
- Option -i interactively ask to delete each file.

Example:

```
$ rm prog1
```

2.1.6 cat - concatenate and display files

```
$ cat [option...] [file ...]
```

If file not specified, take input from the standard input

- Option -c silent about the input files that cannot be accessed for reading.

Example:

```
$ cat prog1
```

```
$ cat
```

```
$ cat prog1 prog1 out >prog1 doc
```

2.1.7 pwd - print working directory

```
$ pwd
```

Display the full pathname for the current directory.

Example:

```
$ pwd
```

```
/user/john
```

```
$
```

2.1.8 cd - change working directory

\$ cd [directory]

If specified, 'directory' becomes the new working directory, else home directory is used.

Example:

```
$ cd      changes to home directory
$ cd .    No change ( . denotes current directory)
$ cd ..   changes to the parent directory
$ cd /    changes to root directory
$ cd dir1 changes to directory dir1 (where dir1 is a subdirectory in the current
          directory)
```

2.1.9 mkdir - make directories

\$ mkdir [-m mode] dirname ...

Creates named directories in system default mode

Standard entries in a directory (., ..) are made automatically.

Owner ID and group ID are set to the process's real user ID and group ID respectively.

- Option -m allows user to define the mode to be used for directory.

Example:

```
$ mkdir dir2
$ pwd
/user/john
$ cd dir2
$ pwd
/user/john/dir2
$
```

2.1.10 rmdir - remove directories

\$ rmdir dirname

Directory must be empty before it can be removed.

Example:

\$ rmdir dir2

2.1.11 chmod - change the access permissions of a file or directory

\$ chmod mode file...

\$ chmod [who] [+/-/=] [permission ...] file ...

- [who] --

a	all users
g	group
o	others
u	user

- [+/-/=] --

+	add
-	remove
=	assigns

- [permission] --

r	read
w	write
x	execute

- Mode in absolute format is based on octal numbers representing the three kinds of access permissions.

4	read
2	write
1	execute

Example:

\$ chmod 754 prog1

\$ chmod ug+x prog1

2.1.12 chown - change owner ID of the files or directories

\$ chown owner file ...

Owner may be a decimal user ID or a login name found in the file /etc/passwd.

This utility is governed by the chown kernel authorization. If it is not granted, ownership can only be changed by root.

Example:

\$ chown tutor test

2.1.13 chgrp - change group ID of the files or directories

\$ chgrp group file

Group may be either a decimal group ID or a group name found in the file /etc/group.
Only the owner or the superuser can change the group ID of a file.

Example:

\$ chgrp bin example

2.1.14 touch - updates access and modification times of a file

\$ touch [-amc] [mmddhhmm[yy]] files

- mm - month
- dd - date
- hh - hour
- mm - minutes
- yy - year

If no time is specified, the current time is used.

- Option -a updates only the access time.
- Option -m updates only the modification time.
- Option -c prevents creating the file if it did not exist previously.

Example:

\$ touch -a test.c

\$ touch -m 1221101395 test.c

2.1.15 ps - get process status

\$ ps [option...]

Display process status information for processes associated with your control terminal.

Examples:

```
(1)    $ ps
        435    ttyp0    00:00:00 login
        456    ttyp0    00:00:02 ksh
        576    ttyp0    00:00:00 ps
```

\$

(2) \$ps -ef

To see the processes that are running

2.1.16 who - determine who is on the system

\$ who [option...][who-file]

Example:

\$ who am I

john ttyp0 Dec 20 16:04

\$

2.1.17 kill - send signal to processes

\$ kill -[option] pid

\$ kill -s [signame] pid

Kill sends signal to the process with process ID as pid. If no option is specified ,it sends signal 15 (software termination signal) to the process.

- Option - 1 Hangup signal.
- Option - 2 Interrupt signal.
- Option - 9 terminate the process.
- Option -15 Software termination signal.

Example:

\$ sleep 50 &

[1] 680

\$ kill -9 680

[1] + killed sleep 50 &

\$

2.2 Some other utilities

2.2.1 head - display the first count lines of file

\$ head [-count] [file ...]

If count is not specified, then ten lines are printed.

If no files are specified, head reads from the standard input.

Example:

```
$ ls -l | head -2
```

2.2.2 tail - display the last count lines of the file

\$ tail [+/-[number][lbc]] [option] file

Copies the last part of the named file to the standard output.

Displays lines of file beginning with +number from the beginning or -number from end.

Number is counted in units of lines (l), disk block size (b), or characters (c).

By default, number is counted in units of lines.

- Option -f enables tail to follow the growth of a file.

Example:

```
$ ls -l | tail +5
```

2.2.3 wc - counts and displays the lines, words, and characters in the files specified

\$ wc [option...] filename...

- Option -l display the number of lines
- Option -w display the number of words
- Option -c display the number of characters (includes whitespace characters).

Example:

```
$ wc prog2
```

```
3      9      60 prog2
```

```
$ wc -c prog2
```

```
60 prog2
```

2.2.4 grep - searches the file for the pattern

\$ grep [option...] pattern [file...]

Display the lines containing the pattern on the standard output.

- Option -c report only the number of matching lines
- Option -l list only the names of files containing pattern.
- Option -v display all lines except those containing pattern.

Example:

```
$ grep -c "the" prog2
```


2.2.5 diff - reports the differences between files

\$ diff [option...] file1 file2

- Option -b ignore trailing whitespaces and compare other strings of space characters.

Example:

\$ diff prog1 prog2

2.2.6 uniq - remove adjacent repeated lines

\$ uniq [option...][+n][-m] [input[output]]

- Option -u display only the lines not repeated in the file named input.
- Option -d display only the lines that are repeated in the file input.
- Option -c precede each line displayed by the number of times it occurs.
- +n ignore the first n characters including whitespace
- -m ignore the first m fields and any blanks.

Example:

\$ uniq -u prog1

2.2.7 tr - translate characters

\$ tr [option...] string1 [string2]

tr is a filter program that doesn't recognize file arguments.

- Option. -c complement the content of string1; that is, all characters except those in string1 are considered.
- Option -d delete all the input characters specified in string1. Any string2 is ignored.
- Option -s compress all strings of repeated output characters that are in string2 into single character.

Example:

\$ tr -s " " " " < prog1

This will convert all number of white spaces between the words in the text into one

2.2.8 cmp - compares two files

\$ cmp file1 file2

Compares two files and if they are different, it displays the byte and the line number of the first difference.

- Option -l prints the byte number and the differing bytes for each difference
- Option -s returns an exit code 0 for identical files, 1 if files are different and 2 if file is inaccessible.

Example:

```
$ cmp prog1 prog2
```

2.2.9 cut - cuts out selected fields of each line of a file

```
$ cut -f list [-d char] [file1 file2 ...]
```

List following `-f` is a list of fields assumed to be separated by a delimiter character.

- Character following `-d` is the delimiter. Default is tab.

Example:

```
$ cut -f1,3 -d "w" prog2
```

2.2.10 paste - merges the corresponding lines of the given files.

```
$ paste file1 file2
```

```
$ paste -d file1 file2
```

- Option `-d` allows replacing tab character by one or more alternate characters.

Example:

```
$ paste prog1 prog2
```

2.2.11 sort - arrange lines in alphabetic or numeric order

```
$ sort [option] file
```

- Option `-d` dictionary order
- Option `-n` arithmetic order
- Option `-r` reverse order

Example:

```
$ ls -l | sort -n
```

2.2.12 find - search for files

```
$ find [path] [expression]
```

Most commonly used expressions:

<code>-mount</code>	Do not descend directories on the file systems
<code>-print</code>	Print the current file name
<code>-exec <i>command</i></code>	Execute command
<code>-user <i>uname</i></code>	File is owned by <i>uname</i>
<code>-type <i>t</i></code>	File has a type of <i>t</i> , such as <i>d</i> for directory and <i>f</i> for file
<code>-name <i>pattern</i></code>	Look for the file name of pattern
<code>-newer <i>file</i></code>	File was modified more recently than <i>file</i>

Example: (1) \$ find /home -name ftp

 This searches the path /home looking for the name ftp.

(2) find . -name "*.c" | xargs grep "pattern"

 This search for a pattern in all *.c files in the current and all the subdirectories

 xargs helps in recursively searching for the pattern in each of the files.

2.2.13 file - used to determine the file type

\$ file [options] files

classify the named files according to the type of data they contain.

Options:

-c	check the format of the magic file (files argument is invalid with -c)
-flist	Run file on the filenames in list
-h	Don't follow symbolic links

Example:

 \$ file msg.c

 Described by the file command as a C Program text.

2.2.14 du - Print disk usage

Prints the number of 512-byte blocks used by each named directory and its subdirectories. Default is the current directory.

Options:

-a	Print usage for all files, not just subdirectories
-r	Print "cannot open" message if a file or directory is inaccessible
-s	Print only the grand total for each named directory

Example:

 \$ du -s /home/nfs/*

To receive the result for the entire "home" directory as a summary

2.2.15 df - Reports the number of free blocks and inodes available on all mounted file systems

\$ df [options] [name]

"name" can be a device name (eg., /dev/dsk/0s9, the directory name of a mounting point (e.g., /usr), a directory name, or a remote resource name (e.g., an RFS/ NFS resource).

Options:

-b	Print only the number of free Kilobytes
-e	Print only the number of free files
-f	Report free blocks but not free inodes
-l	Report only on local file systems
-t	Report total allocated space as well as free space

Example:

```
$ df -t
```

2.2.16 ulimit - Prints the value of one or more resource limits

\$ ulimit [options] [n]

Print the value of one or more resource limits, or, if *n* is specified, set a resource limit to *n*. Resource limits can be either hard (-H) or soft (-S). By default, ulimit sets both limits or prints the soft limit.

Options:

-H	Hard limit. Anyone can lower the hard limit; only privileged users can raise it.
-S	Soft limit. Must be lower than the hard limit.
-a	Print all limits
-c	Maximum block size of core files
-n	Maximum file descriptor plus 1

2.2.17 eval - Forces variable expansion to happen first and then runs the resulting commands

\$ eval args

eval is used in shell scripts, and *args* is a line of code that contains shell variables. The “double scanning” is useful anytime shell variables contain input/output redirection symbols, aliases or other shell variables.

Example:

```
for option
do
    case "$option" in
        # define where output does
        save) out = `> $newfile` ;;
        show) out = `| more` ;;
        esac
done
.....
eval sort $file $out
```

This fragment of a Bourne shell script shows how *eval* constructs a command that is interpreted in the right order.

2.2.18 tput - Prints the value of the terminal capability

\$ tput [options] capname [arguments]

Print the value of the terminal capability *capname* and its associated numeric or string arguments from the terminfo database. *Capname* is a *terminfo* capability such as *clear* or *col*

Exit statuses are :

- 0 when the boolean *capname* is set to true or when a string *capname* is defined;
- 1 when a boolean is false or when a string is undefined
- 2 for usage errors;
- 3 for unknown terminal type;
- 4 for unknown *capname*;

Options:

- Ttype Print the capabilities of terminal type (default is the terminal in use)
- S Read *capname* from standard input (this allows *tput* to evaluate more than one *capname*)

2.2.19 tar - Copy files to or restore files from tape

\$ tar [options] [files]

If any of the *files* are directories, **tar** acts on the entire subtree. Options need not be preceded by -.

Function Options (choose one)

- c create a new tape
- r Append *file* to tape
- t Print the names of *files* if they are stored on tape
- u Add files if not on tape or if modified
- x Extract *files* from tape (if *files* not specified, extract all files)

Options:

- f arch Store files in or extract files from archive *arch* (*arch* is a device name – default is /dev/mt0)
- o Change ownership of extracted files to that of user running program.

Example:

```
$ tar cvf - find . -print > backup.tar
```

This creates an archive of the current directory, and stores it in file backup.tar on the system.

2.2.20 gzip

gzip is an utility used for compression. The gzip files have a .gz extension. Files which have the extension: tar.gz or .tgz are tar files compressed with gzip.

gzip reduces the size of the named files using Lempel-Ziv coding (LZ77). Whenever possible, each file is replaced by one with the extension .gz, while keeping the same ownership modes, access and modification times. If no files are specified, or if a file name is "-", the standard input is compressed to the standard output. Gzip will only attempt to compress regular files. In particular, it will ignore symbolic links.

Compressed files can be restored to their original form using gzip -d.

gunzip takes a list of files on its command line and replaces each file whose name ends with .gz, -gz, .z, -z, _z or .Z and which begins with the correct magic number with an uncompressed file without the original extension. gunzip also recognizes the special extensions .tgz and .taz as shorthands for .tar.gz and .tar.Z respectively.

While compressing, gzip uses the .tgz extension if necessary instead of truncating a file with a .tar extension.

Multiple compressed files can be concatenated.

In this case, gunzip will extract all members at once. For example:

```
gzip -c file1 > abc.gz  
gzip -c file2 >> abc.gz
```

Then

```
gunzip -c abc
```

is equivalent to

```
cat file1 file2
```

In case of damage to one member of a .gz file, other members can still be recovered (if the damaged member is removed).

However, better compression is obtained by compressing all members at once:

```
cat file1 file2 | gzip > abc.gz
```

compresses better than

```
gzip -c file1 file2 > abc.gz
```

If concatenated files are to be recompressed to get better compression,

```
gzip -cd old.gz | gzip > new.gz
```

could be used.

2.2.21 nohup

\$ nohup *command* [*arguments*] &

Continue to execute the named *command* and optional command *arguments* after you log out.

2.2.22 nice

\$ nice [*option*] *command* [*arguments*]

Execute a *command* and *arguments* with lower priority (i.e., be nice to other users)

Option:

- n Run *command* with a niceness of *n* (1-19). Default is 10. Higher *n* implies lower priority. A privileged user can raise the priority by specifying a negative *n* (e.g., -4)

2.3 Regular expressions

A **regular expression** is a string of ordinary and metacharacter which can be used to match more than one type of pattern.

The regular expression character set :

<u>Expression</u>	<u>Significance</u>
ch*	Matches zero or more occurrences of the previous character ch
.	Matches a single character
[pqr]	Matches a single character p,q or r
[c1-c2]	Matches a single character within the ASCII range represented by the character c1 and c2
[^pqr]	Matches a single character which is not a p, q or r
^ptn	Matches the pattern ptn at the beginning of a line
ptn\$	Matches the pattern ptn at the end of a line
ch\{m\}	Matches m occurrences of the character ch
ch\{m,n\}	Matches a range of m to n occurrences (both inclusive) of the character ch
ch\{m,\}	Matches a minimum of m occurrences of character ch
\(ptn\)	Matches the pattern ptn enclosed by \((and \) with the tag \n where n is an integer between 1 and 9

IMPORTANT NOTE : While these characters are special to the command, and some of them to the shell, adequate care has to be taken to ensure that the shell doesn't interfere in affairs which legitimately concern the command. That is why all regular expressions should be single quoted to prevent the Shell pouncing on them.

Examples:

- 1) *Select all Choudhury'es, irrespective of the spelling, from emp.lst*

```
$ grep '[cC]ho[wu]ldh*ury' emp.lst
```

Note that the clause `h*` means that there can be either no `h` or as many `h`'s.

2) Select where salary is 7000 or more (assume salary is the last field)

```
$ grep '[7-9]...$' emp.lst
```

3) How many are directors ?

```
$ grep 'director.*[7-9]...$' emp.lst
```

4) How many directors - the count ?

```
$ grep -c 'director.*[7-9]...$' emp.lst
```

Note that we are searching for a string in the record belonging to class `[7-9]` followed by three more dots (...). The pattern is anchored suitably to the end of the line with `$`. The character sequence `.*` is often embedded in a regular expression to indicate that there can be any number of characters at that point.

Usage of quotes in regular expressions:

Use double quotes: if `[ $v = "first" ]` then

Here if `v` variable doesn't have any value, it will go in else part.

if `[ $v = "first" ]` then

Here if `v` variable doesn't have any value, program will throw syntax error

To delete empty lines from a file :

```
cat <file name> | tr -s '\n'
```

To delete files in a particular directory which are older than some particular time:

```
find <Directory> -name '.*' -mtime +30 -print -exec rm {} \;
```


Assignment - 2

1. Explain how the following commands would work -

`$ cp file1 file2 file3`

Try it out and verify.

2. Explain how the following command would work-

`$ mkdir dir1 dir2 dir3 dir4`

Try it out and verify.

3. What action does the touch command perform?

4. A typical 'who' command displays data in the following format -

`root tty1 Jan8 11:55`

`mary tty12 Jan8 14:16`

How can you sort the fourth column of data in this listing using the 'sort' command ?

5. Write a grep command to find the users who are not using terminal tty2, tty3 or tty4.

6. How can the ls command list-hidden files? What actually constitutes a hidden file in Unix?

7. What is the difference between the two commands below?

(a) `who | tail +1`

(b) `who`

8. Explain why the -r option of rm is dangerous.

9. Draw the tree structure of the file system created by the following commands (assume you are in the directory /usr/office)

`$ mkdir left`

`$ mkdir middle`

`$ mkdir right`

`$ cd left`

`$ mkdir left middle right`

`$ cd ../middle`

`$ mkdir dir dir1 /usr/office/right/dir3`

10. With reference to the last example why is it not possible to issue the command `rmdir /usr/office/right`?
11. Give any three circumstances when a directory file cannot be removed using the `rmdir` command (not including any factors involving file permission settings)
12. Use the command `touch <filename>` on an existing file. Use `ls` and observe what happens. Try the same thing on a non-existent file and observe the results.
13. Use `mv` to rename a set of files `m*` to `t*`. Does it work? Explain the results.
14. Use `kill` to terminate your Shell. Try to explain the results.
15. Create a link to a file. Check the link count and i-node number of both the files. Why is it that both the files have the same i-node number? Use `rm` to remove one of the files. What happens to the link count?
16. List the names of all processes not associated with any terminal.
17. List the lines 5 - 10 of a file.
18. Count the number of files owned by you.
19. List all the filenames and their sizes in ascending order of size.
20. List all the possible users working on more than one terminal.
21. Find all instances of '`fprintf()`' in all the C program source code, resides in your current directory.
22. Devise `egrep` patterns to match -
 - a) Rama or Sita
 - b) Ram Krishna or Shyam Krishna or Gopi Krishna
23. Count the number of lines which has at least one character which is not a space or a tab.
24. Suppose inadvertently a regular expression itself was allowed into a file. How do you locate the line containing it ?
25. How do you locate the lines containing Saxena and Saksena from a file ?
26. Locate all lines beginning with a dot (.) in the file.
27. How would you delete lines containing white space from a file ?

3 FUNDAMENTALS OF UNIX SHELL PROGRAMMING

Keywords in this chapter

Named Variable	Shell	Pipes	Re-direction
Signals	Test Command	Shell Script	Back-quoting
Trap	Error Checking	Exit codes	

3.1 Shell functions

The Shell serves the following functions in a UNIX system.

1. Program execution
2. File name substitution using metacharacters
3. I/O redirection
4. Pipeline hook-ups
5. Environmental control
6. Interpretive programming language

3.2 Shell variables

In shell programming, we deal with three basic types of variables .

1. Positional parameters
2. Special parameters
3. Named variables

3.2.1 Positional parameters

Positional parameters (designated \$1, \$2, \$3,, \$9) acquire their values from the position of arguments at the command line.

Examples:

(i) Illustrating positional parameters

```
$ cat > name
echo Your first name is : $1
echo Your middle name is : $2
echo Your family name is : $3
{ctrl - d}
$
```

\$ chmod u+x name Make it executable

\$ name Jack Maveric Stein Run it.

Your first name is : Jack

Your middle name is :Maveric

Your family name is : Stein

Note that Jack, Maveric, Stein were respectively assigned to \$1,\$2 and \$3

The set command can be used to set values for \$1,\$2,\$3.....

\$ set one two three

\$ echo \$3 \$1 \$2

three one two

(ii) A birthday message

\$ cat birthday

banner Happy Birthday | mail \$1

Make the file Birthday executable

\$ birthday user1 -User1 is mailed the greeting.

3.2.2 Special parameters

Shell assigns the value for the special parameter.

symbol	meaning
\$#	The number of positional parameters while invoking the shell script
\$-	Shell options
\$?	Exit status of the last executed command
\$\$	The process number (PID) of the current shell
#!	The process number of the last background command
\$0	The name of the command being executed
\$*	The list of positional parameters
\$@	Same as \$*, except when enclosed in double quotes

Examples:

(i) Illustrating \$#

\$ cat how_many_args

echo The number of arguments specified is : \$#

```
$ how_many_args a b c d
```

 Run the script

```
The number of arguments specified is : 4
```

*(ii) Illustrating \$**

```
$ cat display_args
```

```
echo The arguments are : $*
```

```
$ display_args UNIX is the best OS
```

```
The arguments are : UNIX is the best OS
```

3.2.3 Named variables

A named variable is a user-defined variable that can be assigned a value within a Shell program.

The value of a named variable can be retrieved by preceding the variable name with a '\$' sign

Example:

```
$ xvar=hai
```

```
$ echo $xvar
```

```
hai
```

Other methods are available for accepting input into named variables.

Use either lower case or upper case characters while naming variables.

3.2.3.1 Using the 'read' command which accepts input interactively

Examples:

(i) To accept input of names and telephone numbers and append to a file named my list

```
$ cat name_list
```

```
echo Enter name:
```

```
read name
```

```
echo Enter phone number:
```

```
read number
```

```
echo $name $number >> mylist
```

(ii) To search for phone numbers in mylist

```
$cat num_please
```

```
echo Enter the name of person:
```

```
read name
```

```
grep $name mylist
```

3.2.3.2 Re-directing the output of a valid Unix command(s) enclosed in back-quotes to the variable i.e., using command substitution

Example:

(i) Finding the early bird

```
earlybird=`who | sort -n +4 | head -1`
```

```
echo The first to login was : $earlybird
```

First entry chosen using `head -1` from the output of `who` command sorted in ascending order on login time, is assigned to the variable `earlybird`.

3.2.3.3 Assign a positional or special parameter to the variable

Example:

(i) Finding the late latif

```
latif=`who | tail -1 | cut -f1 -d" "`
```

```
echo The login time for $latif is : `who | tail -1 | tr -s " " " " | cut -f5 -d" "`
```

3.3 The shell - metacharacters and special symbols

The UNIX shell ascribes special meaning to several metacharacters and recognizes many symbols.

3.3.1 Wild-card substitution (filenames)

symbol	meaning
?	Any one character
*	Any combination of characters (including null) but excluding an initial period
[list]	Any one character from the contained list
[^list]	Any one character not contained in the list

3.3.2 Re-direction and Pipes

symbol	meaning
> file	Make file the standard output
< file	Make file the standard input
>> file	Make file standard output, appending to it if it already exists

<< word	Take shell input up to the first line containing word or up to end of file
n>file1	Make file1 as the output for file descriptor n
1>&2	Redirect standard output to standard error
command1 command2	Redirect the standard output of command1 to the standard input of command2

Note: If the output of a command is to be appended to a file and the file is not writable, then error occurs. Therefore check the status of file before executing this command

3.3.3 Other command line notations

symbol	meaning
\	Continue command on next line
&	Run preceding command in background mode
;	Command separator
spaces, tabs and newlines	Word separators
()	Command grouping
cmd1 && cmd2	Run cmd2 if cmd1 succeeds
cmd1 cmd2	Run cmd2 if cmd1 fails

3.3.4 Quoting

symbol	meaning
\	Interpret the following character literally
' '	Interpret the enclosed characters literally
" "	Interpret the enclosed characters literally except for \$ combinations and *, which will be expanded into the corresponding parameters and filename values, also command line substitutions takes place

3.3.5 Command substitution

symbol	meaning
'command'	Substitute the output of command for the expression inside back-quotes.
# or :	Interpret the rest of the line as a comment

3.3.6 Parameter substitution

\${parameter-word}	Expression has parameter's value if any; otherwise has the value word
\${parameter=word}	Expression has parameter's value if any, otherwise has the value word and sets parameter to word
\${parameter+word}	If parameter is set, then substitutes with word, otherwise substitutes with nothing
\${parameter?mesg}	Has parameter's value if any, otherwise substitutes mesg and exits from shell

3.3.7 Shell commands

:	The null command, which does nothing but can have active arguments
. file	Run commands in file without creating new shell

3.4 Looping and making choices

3.4.1 The for loop

The general form of the **for** loop in Bourne shell is:

```
for variable-name in value1 value2 ....  
do  
<commands>  
done
```

NOTE : The combination, **for i in \$*** can be replaced by just **for i**

Examples:

(i) Illustrating for

```
for VAL in 1 2 3 4  
do  
echo $VAL  
done
```


The following are some important methods for generating values for a for loop

(i) List the values explicitly after in

```
for name in Dutt Shah Muneem
```

(ii) Take the values from shell script arguments using \$*

```
for i in $*   or   for i
```

(iii) Take the filenames from a directory as values using * and ?

```
for file in *  
  
do  
  
cat $file  
  
done
```

(iv) Take the values from a named variable using the read command

```
echo Hi ! What's your name\  
  
read NAME  
  
for N in $NAME  
  
do  
  
echo $N  
  
done  
  
echo is quite a nice name.
```

(v) Take the values from the output of a command using the back-quote mechanism

```
for i in `cat animals`      # animals is a file containing names of animals one per line  
  
do  
  
echo I know an old lady who swallowed a $i.  
  
echo Swallowed a $i ?  
  
echo Swallowed a $i!  
  
done
```

3.4.2 The case statement

```
case value in
    choice1) commands ;;
    choice2) commands ;;
    ....
    ....
esac
```

choice1 and *choice2* are labels that identify potential choices of action; if the *value* portion has the value *choice1*, then the commands following the *choice1* label are executed.

Note: The operator “|” is interpreted as “or” operator in *case* statements.

Example:

A menu -

\$ cat menu

echo Please enter the number of the command you wish to run :-

echo '1 date 2 who'

echo '3 ls 4 pwd'

read choice

case \$choice in

1) date ;;

2) who ;;

3) ls ;;

4) pwd ;;

*) echo That wasn't one of the choices ! ;;

esac

3.4.3 The while loop

(a) The general form of the while loop is

```
while control command
do
    <commands>
done
```

Note: A zero (0) exit status denotes success

Example :

(i) Illustrating the UNIX while

```
# grit -- grep and quit when word not found
# Usage : grit word file(s)
word=$1
shift
while grep $word $1 > /dev/null
do
shift
done
echo First file not containing $word is \1
$ grit UNIX report.[1-9]
```

Suppose the file report.2 does not contain the word UNIX

\$ First file not containing UNIX is report.2

This program has a bug. Spot it. Discuss.

The UNIX While is different from the usual While - How? Comment.

*Realize the need for the **test** command*

3.4.4 The until loop

```
until control command
do
    <commands>
done
```

NOTE : While both loops i.e. *while* and *until* are conditional, while loop runs until a control command fails, and the until loop runs until a control command succeeds.

Example:

(i) Watching for a file

```
# isdone -- report when file $1 is created
# Usage : isdone filename
until ls | grep $1 > /dev/null
do
sleep 30
done
echo file $1 is here
```

3.4.5 The if statement

```
if control command
then
    commands
fi
```

Note : The Shell if differs from other language ifs in that it is concerned with the status of the control-command (success => exit status of 0) rather than with variable value comparisons like $x > 3$ etc.,

Example:

(i) Removing a duplicate file

```
# Usage : clean file1 file2
```

```
if cmp -s $1 $2
then
rm $2
fi
```

Note : The conditional operator && simulates 'if' 'then' 'fi'-

```
$ cmp -s $1 $2 && rm $2
```

3.4.6 if.....then.....else statement

```
if control command
then
    <commands>
else
    <commands>
fi
```

Example:

(i) Enhancing 'clean'

```
if cmp -s $1 $2
then
rm $2

echo The $2 file was a duplicate and has been removed
else
echo The files $1 and $2 are not the same.
fi
```

3.4.7 if then.....elif then.....else statement

```
if control command1
then
```

```
        <commands>
    elif control command2
    then
        <commands>
    else
        <commands>
    fi
```

Example:

(i) Find if 'name' in atype or btype file

```
# Usage : a_or_b name
if grep $name atype > /dev/null
then
    echo $name is Type A
elif grep $name btype > /dev/null
then
    echo $name is Type B
else
    echo $name was not found
fi

$ a_or_b lancelet           - filename lancelet
$ lancelet is Type A        - if lancelet is found in file atype
```

3.5 The test command

The test command returns an exit status of 0 if the test succeeds and 1 otherwise. It can be used with an advantage in Shell scripts in the following situations.

Note: The test command produces no output but it produces an exit status which can be checked by using the shell special variable \$?

Note that the multiple piped commands will give exit status of the last command only.

Examples:

```
$ cat=pussie
```

```
$ test $cat = pussie
$ echo $?
0                                0 implies success, the value of the variable cat is pussie
$ test $cat = cussie           (Note spaces on both sides of =)
$ echo $?
1                                1 implies failure, the value of the variable cat is not cussie
$
```

3.5.1 File checking

<i>r file</i>	True if file exists and is readable
<i>w file</i>	True if file exists and is writable
<i>f file</i>	True if file exists and is not a directory
<i>d file</i>	True if file exists and is a directory
<i>s file</i>	True if file exists and has a size > 0
<i>x file</i>	True if file exists and is executable
<i>t [fildes]</i>	True if the file indicated by file descriptor <i>fildes</i> (1 by default) is open and a terminal.

Examples:

(i) Check to see if a file is readable before copying it

```
$ cat checkf
if test -r $1
then
cp $1 $HOME/project_x
fi
```

(ii) Testing based on 'file descriptor' numbers

```
$ test -t ; echo $?
0                                - terminal was standard output
$ test -t >save ; echo $?
                                - redirect standard output
$                                - terminal not standard output
```

3.5.2 String checking

<i>string1 = string2</i>	True if the strings are the same
<i>string1 != string2</i>	True if strings differ
<i>-n string</i>	True if the string has non-zero length
<i>-z string</i>	True if the string has zero length
<i>string</i>	True if the string is not the null string (an undefined variable is null)

Note: Spaces around the operators are a must.

3.5.3 Numeric comparisons

<i>n1 -eq n2</i>	True if the integers are equal
<i>n1 -ne n2</i>	True if integers are not equal
<i>n1 -gt n2</i>	True if n1 is greater than n2
<i>n1 -lt n2</i>	True if n1 is less than n2
<i>n1 -ge n2</i>	True if n1 is greater than or equal to n2
<i>n1 -le n2</i>	True if n1 is less than or equal to n2

3.5.4 Logical operations

<i>!</i>	Negates the following expression
<i>-a</i>	Binary 'and' operator
<i>-o</i>	Binary 'or' operator

Examples:

(i) Test files for multiple permissions

<i>\$ test ! -r flam</i>	- true if flam is not an existing, readable file
<i>\$ test -r flam -a -w flam</i>	- true if flam is readable and writable
<i>\$ test -r flam -o -w flam</i>	- true if flam is readable or writable
<i>\$ test \ (-r flam -a -w flam \) -o \ (-r flim -a -w flim \)</i>	

Note: [*Sans* = pickle] same as test *Sans* = pickle i.e., [] is same as test

3.6 The expr command

The expr command evaluate its arguments as an expression and writes the result on to the standard output. It is useful for performing variable arithmetic and other variable manipulations within a shell script. The expression must stick to the following rules:

1. The terms of expression must be separated by spaces.
2. Individual parameters within an expression may need to be quoted or escaped.
3. Strings containing blanks or other special characters should be quoted.

Logical and mathematical operators can be used in `expr`.

<code>arg1 arg2</code>	Returns <code>arg1</code> if it is neither null nor zero, otherwise <code>arg2</code> . Here <code>arg2</code> is a pattern
<code>arg1 & arg2</code>	Returns <code>arg1</code> if both arguments are not null or nonzero, otherwise returns zero.
<code>arg1 : arg2</code>	Compares the <code>arg1</code> with <code>arg2</code> and returns the number of matching characters, otherwise returns 0
<code>arg1 { =, ==, <, <=, } arg2</code>	Compares the arguments and returns 1 if the expression is true and 0 if <code>></code> , <code>>=</code> , <code>!=</code> the expression is false.
<code>arg1 { +, -, *, /, % } arg2</code>	Evaluate the expression (addition , subtraction, multiplication, division or reminder of the integer-valued expressions) and returns the result.

Examples:

Create a counter in a shell script

```
$ cat counter
a=2
while test $a -lt 10      or      while [$a -lt 10]
do
a=`expr $a + 1 `
echo -n $a" "
done
$ counter
2 3 4 5 6 7 8 9 10
$
```

3.7 Error checking in shell programs

The hallmark of good software is that it incorporates foolproof error checking - UNIX shell programming offers the programmer several artifacts to build sound error checking in the shell scripts. A typical example will bring out the intricacies involved.

Here is a program called *posses* that makes a copy of given file and places the file in the HOME directory of the user under the name *file.login_name*.

Example:

```
$ cat posses
filename=$1
set 'who am i'
cp $filename $filename.$1 # $1 is now the user-id of the user assigned to $1 by set
```

This program has no immunity under the following conditions:

1. The user may use the wrong number of arguments
2. The argument may not represent a file
3. The user may not have permission to copy the file

The task is to develop code to guard the user against the above error conditions

```
$ cat posses

# Copy a file to users home directory and tag it with users login name along
with error checks :

# 1) Check for proper number of arguments

case $# in
1) ;;
0) echo .Usage : $0 filename 1>&2
exit 1 ;;
*) echo Warning: Only the first argument is used 1>&2 ;;
esac

# 2) Check for file existence and copy permission

if test ! -f $1
then
echo $0: File $1 not found 1>&2
exit 2
elif test ! -r $1
then
echo $0: File $1 found but not readable 1>&2
exit 3
else
```

```
# No errors now, proceed with main task

filename=$1

set 'who am i'

cp $filename $HOME/$filename.$1

fi
```

COMMENTS :

1. Error checking makes programs more *robust* but increases code size. So, one can compromise depending on the need.
2. The exit command : Terminates a script and gives it an exit status which is simply the integer argument given to exit. Zero (0) exit status implies success.
3. Error messages prefixed with \$0 rather than the name of a script file.
4. Redirection 1>&2 ensures that, despite standard output (file descriptor 1) being redirected or piped - as can happen when the shell script is used in another shell script - the standard output (here error messages from Echo) is not lost because it is redirected into standard error (file descriptor 2). Typically standard error is the terminal.

3.7.1 Signals - the trap command

UNIX provides the *trap* command which allows the user to alter the effects of certain events that generate signals, which generally tend to halt a process:

- The *hang-up* signal is denoted by code number 1.
- The *interrupt* (*Ctrl-C*) signal is denoted by code number 2.
- The *terminate* (*by kill command*) is denoted by code number 15.

The general form of the *trap* command

trap *command signal list*

The *trap* command, once it has been read in, lies in wait for those signals mentioned in the signal list. When such a signal shows up, the command is performed instead of the consequence of the signal that would have been to halt the process.

As an illustrative example, we will consider a script that will search a file for individual lines that contain all the given search words

Examples:

```
# mgrep - search for multiple words per line
# first argument is file to be searched
# remaining arguments are the words sought
# Usage: mgrep file words

file=$1

shift

temp1=/tmp/mgrep1

temp2=/tmp/mgrep2

trap 'rm -f $temp1 $temp2 ; exit 1' 12 15

for word
do

grep $word $file > $temp1

cp $temp1 $temp2

file=$temp2

done

cat $temp1

rm -f $temp1 $temp2
```

COMMENTS:

1. The rm command is used for the purpose of house keeping - it removes the temporary files after the main program has run.
2. The rm would have failed, but for the *trap*, if one the signals discussed above were to be triggered before the end of the run. The exit command terminates the process only after the housekeeping, it also ensures that the script does not continue after rm.
3. A signal can also be simply ignored and the script allowed to continue without interruption by the following form of the trap
trap `` 1

3.8 The export command

A Shell variable can be *exported* so that it becomes part of the *environment*. A process can then access and use any of the variables in its environment.

Note : An exported variable becomes part of the environment of the current shell and of all processes *descending* from it. Also, it is important that to make a variable accessible to all subshells, it should be defined and exported from the login shell itself.

Example:

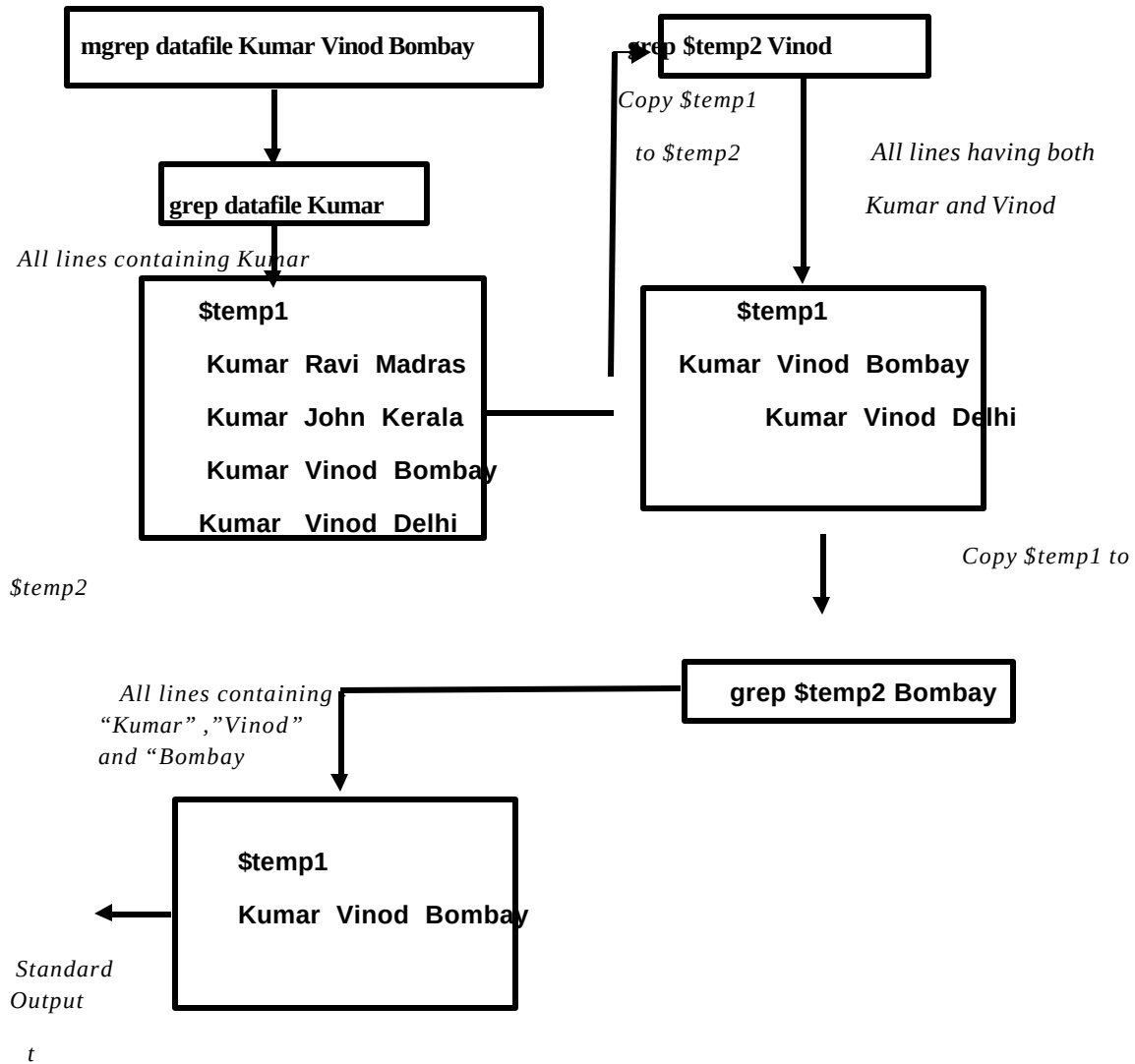
```
$ export TERM          TERM is exported.
```

A variable if not exported will not be visible to the subshells.

Example:

```
$ x=2
$ sh
$ echo $x
.....
$ exit
$ export x
$ sh
$ echo $x
2
$ x=3
$ echo $x
3
$ exit
$ echo $x
2
```

If the value of a variable is changed in the subshell it will not be reflected in the parent shell

The algorithmA multiple grep search

3.9 Environment Setting

This script xxx_env.ksh shows some environment setting required for executing database related scripts. This script defines the environment variables for a Project. It is mandatory to run this script before executing any other scripts that Infosys uses.

```
#!/bin/ksh
# *****
# Application : XXX
# File       : xxx_env.ksh
# Description : This script defines the environment variables for
#             XXX Project.
#             It is mandatory to run this script before executing any other
#             scripts that Infosys uses.
#
# Usage      : xxx_env.ksh
#
# Modification History:
#
# Date      Person      Remarks
# -----
# dd mmm yy H.P. Ramesh   Initial version.
# *****/

#
# **** IMPORTANT: We shouldn't do an "exit" from this script!!
# **** IMPORTANT: This script should use Bourne Shell language
#             Use back-quotes instead of $( )
#             Use var=value; export var instead of export var=value
#             No ksh-only features to be used
#

debug_mode=0
if [ -z "$xxx_ENV_EXECUTED" -o "$1" = "-f" ]
then
    if [ "$1" = "-f" ]
    then
        echo Setting up environment for xxx
        debug_mode=1
        shift
    fi
```

```
xxx_ENV_EXECUTED=1
export xxx_ENV_EXECUTED

progrname=`basename $0`
rc=0

xxx_GS_LOCATION=NYK
export xxx_GS_LOCATION
xxx_ENV=dev
export xxx_ENV
if [ ! -z "$1" ]
then
    host=$1
else
    host=`hostname`
fi

#
# OS specific settings
#
xxx_OS_ARCH=`uname -r | cut -c1`
xxx_OS_ARCH=`echo "+++Sun+Sol" | cut -f$xxx_OS_ARCH -d+`
export xxx_OS_ARCH
case "$xxx_OS_ARCH" in
    Sun)
        PATH=$PATH:/usr/ucb
        xxx_MAIL_CMD=/usr/ucb/mail
        ;;
    Sol)
        xxx_MAIL_CMD=mailx
        ;;
    *)
        xxx_MAIL_CMD=mailx
        ;;
esac

export xxx_MAIL_CMD
export PATH

#
# machine/environment specific settings
#
case "$host" in
nnnn)
    SYBASE=/home/sybase
    xxx_ENV=dev
    xxx_HOME=/disk1/gspara/gsfddpara/xxx
    xxx_GS_LOCATION=NYK
    xxx_GS_DESK_HOME="/disk1/gspara/gsfddpara/desks"
```



```
;;
nhctp*|NYK)
    SYBASE=/opt/sybase
    xxx_ENV=gs_prod
    xxx_HOME=/home1/para_p/xxx
    xxx_GS_LOCATION=NYK
    xxx_GS_DESK_HOME=/home/desks
    ;;
nhctd*|chagall)
    SYBASE=/opt/sybase
    MAIL=/var/spool/mail/padmar
    export MAIL
    xxx_ENV=gs_dev
    xxx_HOME=/home/padmar/xxx
    xxx_GS_LOCATION=NYK
    xxx_GS_DESK_HOME=/home/desks_prod
    ;;
lnctd*)
    SYBASE=/opt/sybase
    xxx_ENV=gs_dev
    xxx_HOME=/home/para_m/xxx
    xxx_GS_LOCATION=LDN
    xxx_GS_DESK_HOME=/home/desks_prod
    ;;
lnctp*|LDN)
    SYBASE=/opt/sybase
    xxx_ENV=gs_prod
    xxx_HOME=/home/para_m/xxx
    xxx_GS_LOCATION=LDN
    xxx_GS_DESK_HOME=/home/desks
    ;;
tkfwd*)
    #
    # This is not used... ignore
    #
    SYBASE=/opt/sybase
    xxx_ENV=gs_dev
    xxx_HOME=/home/para_u/xxx
    xxx_GS_LOCATION=TKO
    xxx_GS_DESK_HOME=/home/desks_prod
    ;;
tkfwp*|TKO)
    SYBASE=/opt/sybase
    xxx_ENV=gs_prod
    xxx_HOME=/home/para_m/xxx
    xxx_GS_LOCATION=TKO
    xxx_GS_DESK_HOME=/home/desks
    ;;
*)
```

```
#
# by default, we assume NY development machine
#
SYBASE=/opt/sybase
xxx_ENV=gs_dev
xxx_HOME=/home/padmar/xxx
xxx_GS_LOCATION=NYK
xxx_GS_DESK_HOME=/home/desks_prod
;;
esac

export xxx_ENV
export xxx_HOME
export xxx_GS_LOCATION
export xxx_GS_DESK_HOME

xxx_SCCS=$xxx_HOME/sccs
xxx_SCCS_MIRROR=$xxx_HOME/sccs_mirror
xxx_SCRIPTS=$xxx_SCCS_MIRROR/scripts
xxx_SQL=$xxx_SCCS_MIRROR/sql
xxx_DDL=$xxx_SCCS_MIRROR/ddl
xxx_SRC=$xxx_SCCS_MIRROR/src
xxx_CONFIG=$xxx_SCCS_MIRROR/static_data
xxx_STATIC_DATA=$xxx_SCCS_MIRROR/static_data
xxx_BIN=$xxx_HOME/bin
xxx_WORKDIR=$xxx_HOME/workdir
xxx_BATCH_ANALYSIS=$xxx_HOME/batch_analysis

export xxx_SCCS
export xxx_SCCS_MIRROR
export xxx_SCRIPTS
export xxx_SQL
export xxx_DDL
export xxx_SRC
export xxx_CONFIG
export xxx_STATIC_DATA
export xxx_BIN
export xxx_WORKDIR
export xxx_BATCH_FAILURES

#
# COB date
#
cob_date_file=$xxx_GS_DESK_HOME/common/controls/batch_date_day
if [ -f $cob_date_file ]
then
    #
    # COB date is taken from the first line
    # Date format is: MM/DD/YY
```

```
# Here we hardcode the century as "20"
#
cob_date_line=`head -1 $cob_date_file`
century="20"
xxx_COB_MM=`echo $cob_date_line | cut -f1 -d^`
xxx_COB_DD=`echo $cob_date_line | cut -f2 -d^`
xxx_COB_YYYY=$century`echo $cob_date_line | cut -f3 -d/ | cut -f1 -d^`
export xxx_COB_MM
export xxx_COB_DD
export xxx_COB_YYYY
fi

#
# For Sybase
#
export SYBASE

#
# For job execution
#
xxx_JOB_SCRIPTS=$xxx_SCRIPTS
xxx_JOB_LOG=$xxx_HOME/jobs/log
PSL_WORKDIR=$xxx_WORKDIR
export xxx_JOB_SCRIPTS
export xxx_JOB_LOG
export PSL_WORKDIR

#
# For notification
#
xxx_JOB_MAILING_LIST="rameshhp@infy.com rohitjha@infy.com nitika_kapur@infy.com
sanjay_jain@infy.com vipin_cherian@infy.com apurbam@infy.com kvkrishnareddy@infy.com ranjanaich@infy.com
nileshm_pisal@infy.com srinivasa_edara@infy.com"
export xxx_JOB_MAILING_LIST
xxx_JOB_FAILURE_CONTACT="rameshhp@infy.com rohitjha@infy.com"
export xxx_JOB_FAILURE_CONTACT

#
# For feed monitor
#
xxx_COMMON_FEEDSDIR=$xxx_GS_DESK_HOME/common/feeds
xxx_PRICING_FEEDSDIR=$xxx_GS_DESK_HOME/price/feeds
xxx_MAINPNL_FEEDSDIR=$xxx_GS_DESK_HOME/common/feeds/ccpnl
xxx_ROLL_DATE_LOG=$xxx_GS_DESK_HOME/maint/logs/PARA-roll_date-getnewday.log
xxx_FEEDS_LOGPATH=\
"$xxx_GS_DESK_HOME/feedval/logs:"\
"$xxx_GS_DESK_HOME/ctlcore/logs:"\
"$xxx_GS_DESK_HOME/price/logs:"\
"$xxx_GS_DESK_HOME/ccpnl/logs:"\
```

```
"$xxx_GS_DESK_HOME/ctlcore/logs/last.logs"
```

```
export xxx_COMMON_FEEDSDIR
export xxx_PRICING_FEEDSDIR
export xxx_MAINPNL_FEEDSDIR
export xxx_ROLL_DATE_LOG
export xxx_FEEDS_LOGPATH
```

```
#
# set the PATH such that all relevant directories are part of it.
#
PATH=$PATH:$xxx_SCRIPTS:$xxx_BIN:$SYBASE/bin
export PATH
```

```
fi
```

```
# !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
#
# **** IMPORTANT: we shouldn't do an "exit" from this script!!
#
# !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
```

3.10 Full Backup Script

full_backup takes the full backup of project development directories.

```
#!/bin/ksh
#
# full_backup: takes the full backup of PPP development directories
#
# by HPR, DATE
#
```

```
BKPDIR=$PPP_HOME/backup
today=`date +"%Y%m%d%H%M%S"`
backupfile=$BKPDIR/full_PPP$today.tar
listfile=$BKPDIR/full_PPP$today.list
cmdfile=$BKPDIR/full_PPP$today.cmd
```

```
cd /ampere/ppp
find sccs_mirror -follow -type f -a -perm -g+r -print > $listfile
find sccs -follow -type f -a -perm -g+r -print >> $listfile
find Dev1 -follow -type f -a -perm -g+r -print >> $listfile
find Dev2 -follow -type f -a -perm -g+r -print >> $listfile
find Dev3 -follow -type f -a -perm -g+r -print >> $listfile
```

```
find smanoj -follow -type f -a -perm -g+r -print >> $listfile
find inbound_outbound/static_data -follow -type f -a -perm -g+r -print >> $listfile

#
# take the latest jadeBackup files - on a weekly basis
#
curr_day=$(date | cut -f1 -d' ')
if [ "$curr_day" = "Sun" ]
then
    find $(ls -1td $BKPPDIR/jadeBackup/* | head -1) -print >> $listfile
fi

echo "tar cvf $backupfile \" > $cmdfile
#
# note: the following sed command puts the file names within quotes to
# protect the file names with blanks in them.
#
sed -e 's/^/' -e 's/$/' \" $listfile | grep -v \"\\.sh_history\" >> $cmdfile
echo >> $cmdfile
ksh $cmdfile
stat=$?
rm -f $listfile $cmdfile
if [ $stat -ne 0 ]
then
    echo "Full SCCS Backup failed due to tar error"
    echo "Login to ampere and check" | mailx -s "Full SCCS Backup Failed" xyz@infy.com abc@infy.com
fi

compress $backupfile
# send the file to mac
cd /ampere/PPP/backup
echo > backup.stat
backupfile_s=`basename ${backupfile}.Z`
ftp << eof
    open fgh.com
    cd PPPBackup
    bin
    put $backupfile_s
    put backup.stat
    ! rm backup.stat
    get backup.stat
    bye
eof
if [ ! -f backup.stat ]
then
    echo "Login to ampere and check" | mailx -s "Backup To Mac Failed - $status" xyz@infy.com abc@infy.com
fi

exit 0
```

j

3.11 Background Processing

failure_daemon_client.ksh script pulls the batch failure information from machine and sends an email to infy account. It should be run as a daemon process.

```
#!/bin/ksh
# ****
# Application : XXX
# File       : failure_daemon.ksh
# Description : This script pulls the batch failure information from GS machine
#             and sends an email to infy account.
#             It should be run as a daemon process.
# Notes      :
#             1. The password for GS login has been hardcoded here!!
#
# Modification History:
#
# Date      Person      Remarks
# -----
# dd mmm yyyy H.P. Ramesh      Initial version.
# ****/

. $xxxx_SCRIPTS/XXX_shell_lib.ksh

DELIMITER="|"
psl_start "$0"

    cd $xxxx_WORKDIR
    while :
    do
        echo "$(date): pulling failure data from GS"
        ftp -i 204.4.55.101 << EOF-FTP
            user padmar@nhctdc187.fw.gs.com BackFromHell
            cd xxxx/workdir
            mget XXX_failures*.dat
        EOF-FTP

        oldIFS=$IFS

        for file in XXX_failures*.dat
        do
            echo ==== $file ====
            cat $file
        done

        sleep 200
    done
```

psl_end S

failure_daemon_server.ksh script is scheduled as a daemon on machine which keeps pulling the failure information periodically into a file. A client daemon running from the Infy machine will pull the failure information across and send a mail to Infy.

```
#!/bin/ksh
# *****
# Application : XXX
# File       : failure_daemon_server.ksh
# Description : This script is scheduled as a daemon on GS machine which keeps
#               pulling the failure information periodically into a file.
#               A client daemon running from the Infy machine will pull
#               the failure information across and send a mail to Infy.
#
# Usage      : failure_daemon_server.ksh [-L location]
#
# Modification History:
#
# Date      Person      Remarks
# -----
# dd mmm yyyy H.P. Ramesh      Initial version.
# *****/
#

. $xxxx_SCRIPTS/XXX_shell_lib.ksh
progrname=$(basename $0)
DELIM="|"

####
#### main processing
####

psl_start "$0"

psl_env_check xxxx_WORKDIR

USAGE="$progrname [-L location] [desk]"
psl_getopt "-L:" "$*"
if [ $? -ne 0 ]
then
    echo $USAGE
    exit_if_error F "Invalid arguments passed to $progrname"
fi

location=${arg_L:-${xxxx_GS_LOCATION:-NYK}}

case "$location" in
```

```
    NYK)
        autosys_db=PRDSCHED2
        input_timings_file=${arg_d:-$xxxx_STATIC_DATA/batch_timings_nyk.dat}
        export AUTOSERV=${arg_A:-CNT}
        ;;
    LDN)
        autosys_db=CTLLDAUTOSYS
        input_timings_file=${arg_d:-$xxxx_STATIC_DATA/batch_timings_ldn.dat}
        export AUTOSERV=${arg_A:-PAR}
        ;;
    *)
        echo "Invalid location: '$location' - value must be NYK or LDN"
        echo $USAGE
        exit_if_error F "Invalid location '$location' passed to $progname"
        ;;
esac

echo "$(date): Using AUTOSERV=$AUTOSERV"
echo "$(date): Using autosys database '$autosys_db'"

if [ ! -z "$getopt_arg_list" ]
then
    set $getopt_arg_list
fi

if [ $# -gt 1 ]
then
    echo $USAGE
    exit_if_error F "Invalid arguments passed to $progname"
fi

psl_mktmp query_file prefix=qry suffix=sql
check_write_access "$progname" "$query_file"
#
# Valid Autosys status values:
# 1 - RUNNING
# 3 - STARTING
# 4 - SUCCESS
# 5 - FAILURE
# 6 - TERMINATED
# 7 - ON_ICE
# 8 - INACTIVE
# 9 - ACTIVATED
# 10 - RESTART
# 11 - ON_HOLD
# 12 - QUE_WAIT
# 13 - DELETED
cat > "$query_file" << --EOF_QUERY
select
```



```
'FAILURE' + '$DELIM' + job.job_name + '$DELIM' +
convert(char,dateadd(second,convert(float,last_end),convert(datetime,"Dec 31 1969
8:00PM")),109)
from
    job,
    job_status,
    intcodes
where
    job.joid = job_status.joid and
    job_status.status = intcodes.code and
    intcodes.fld = "status" and
    job_status.status = 5
--EOF_QUERY

echo "go" >> "$query_file"

psl_mktmp query_results prefix=query_res suffix=dat
check_write_access "$progname" "$query_results"
check_write_access "$progname" $xxxx_WORKDIR/XXX_failures.dat

while :
do
    echo "$(date): Pulling failure information from Autosys database"
    psl_run_query "$autosys_db" autosys autosys "$query_file" "$query_results"
    sed 's/^[ ]*/' "$query_results" | grep "^FAILURE" > $xxxx_WORKDIR/XXX_failures.dat

    sleep 300
done

rm -f \
    "$query_file"

psl_end S
```

3.12 Guidelines for Unix shell scripts

- ✓ Use korn-shell as the standard as it gives the best scripting features.
- ✓ Use all uppercase for shell variables shared across processes.
- ✓ Use all lowercase for local or temporary shell variables.
- ✓ Write only one statement per line.
- ✓ Do error checking for every statement.

- ✓ Don't use recursion.
- ✓ Don't use relative paths in file names.
- ✓ Write shell functions for frequently performed tasks.
- ✓ Don't assume current directory.
- ✓ Don't use mixed case file names or shell variable names.
- ✓ Enclose shell variable expansions in double quotes always, especially in "if" statements and argument lists.
- ✓ Don't assume access permissions on files and directories.
- ✓ While capturing output from a loop, redirect the output of the loop instead of individual statements.
- ✓ Don't make assumptions on the output of some of the commands like 'ps', 'df' etc.
- ✓ Handle interrupts using the 'trap' command.
- ✓ Use unique names for temporary files.
- ✓ Clean up the temporary files on exit.

Assignment -3

1. Write a Shell script that runs in the background and informs you when a person logs onto the system.
The user-id of the person is to be passed as a command line argument.
2. Write a Shell script to reverse a string. The string is to be passed as a command line argument.
3. Write a Shell script that cleans up a directory by removing all duplicate files in a directory.
Duplicates are those that have identical contents. (Note: To avoid a lot of grief, do not use rm in initial untested versions of your program)
4. Improve the above script such that if there are two files filea and fileb, fileb is not be checked against filea if the check of filea against fileb has been done.
5. Improve the above script to clean up all sub-directories under a directory specified on the command line.
6. Point out the error in the following Shell Script

```
echo What day is today ?  
read $day1  
echo day1 is a good day as any
```
7. Write a Shell script that requests your age and then echoes it along with a suitable comment.
8. Write a Shell script that prints out date information in the following format
12:33:51 PDT Sat 15 Oct 1996
9. Write a Shell script that will take a filename as a command line argument, and in the background use the command spell to check the spelling in that file then place the output of spell in a file whose name would be the original filename prefixed with sp.
10. Explain the command
\$ cat meat 1> pluto
11. List all the users who have logged in more than once.
12. List all the files older than a specified date. The date is to be passed as a command line argument.
13. List all the filenames in which the number of unique words is greater than a specified number. The number is to be passed as a command line argument.

- 14. List all the lines, of a file, in which words are repeated. The filename is to be passed as a command line argument.*

15. Suppose the only variables (other than the standard shell variables) defined in a Shell are as follows

fruit=orange

veg=brinjal

What would be the output from the following commands-

- a) echo My favorite fruit is \${fruit-tomato}*
 - b) echo Your favorite fruit is \${dryfruit-grapes}*
 - c) echo Her favorite veg is \${veg+carrot}*
 - d) echo His favorite veg is \${cutveg+cucumber}*
16. Write a Shell script that takes an indefinite number of files names as arguments and places the content of them in a destination file.
- Write a Shell script that presents a multiple choice question, gets the users answer, and reports back whether the answer is right, wrong or not one of the choices.*
17. Write a script which will accept the word OK as an answer regardless of upper-case or lower-case letters are being used anywhere in the word.
18. Write a script that takes a command line argument and reports on whether it is a directory, a file or something else.
19. Write a script that asks for the capital of India and repeats the question until the user gets it right.
20. Write a script that accepts the user into Oby gym if his or her weight lies in the range 75 - 120 Kgs.
21. How would you modify a script so that it would quit if more than one argument were provided ?
- (Hint : Assume the exit is through an exit statement)*
22. How can you find out the exit status of a Shell script ?
23. Suppose the shell variable namea is set to your first name and that nameb to your last name. Give a command that assigns both names (with a separating space) to a single shell variable called name.
24. Write a script that takes a command line argument of Kilograms and, by default, converts that number to pounds. Also provide a -t option to convert Kgs to tones instead and a -q option to convert Kgs to quintals.
25. Print all filenames in which the names are alphabetically greater than a specified string. The string is to be passed as a command line argument.

4 THE VI EDITOR

Keywords in this chapter

Command mode	Insert / Edit mode	Ex Esc / Colon (:) mode
Edit commands	Marking and Yanking	Set options
User defined registers	Key mapping	Global substitution
Pattern searching	Macros	.exrc file

There are primarily three types of editors available under Unix

1. ed is a line editor comparable to EDLIN in DOS.
2. vi is a full screen editor - vi stands for visual.
3. ex is a line editor compatible to vi

4.1 vi : Cautions

1. Very user unfriendly - no help.
2. Same keystrokes can have more than one meaning depending on current mode. Also, the key combinations are in no way mnemonic.
3. Fanatically case-sensitive

4.2 vi : Salient features

1. vi is omnipresent - in all Unix systems.
2. vi is fast.
3. vi got powerful UNDO features.

4.3 vi : The basics

1. vi and ex commands compatible with each other but ex is a line editor only.
2. All editing is done in a buffer i.e., on a copy of original, until the work saved.
3. vi works in 3 different modes :
 - Command mode - All keys are interpreted as commands.
 - Insert or edit mode - Keys are echoed in edit buffer.
 - The ex Escape or The Esc colon (:) mode - Keys are interpreted as commands and echoed in the command line at the bottom of the vi screen.
4. Bottom most line in vi is the command line where all commands in Esc: mode and messages are displayed.
5. There is no error messages in vi - only beeps, beeps and beeps! (PC speaker vital!).
6. Never press any control or special keys - no arrow keys for cursor movement on most terminals!

4.4 vi : Important keys

<u>KEY</u>	<u>DOES</u>
Esc	Returns vi to command mode and cancels partially formed commands.
RETURN/ENTER	Executes commands entered in ex Esc mode and starts a new line in Ins mode. In cmd mode, it takes the cursor to next line.
:	To enter ex Esc mode - the command should appear at the command line preceeded by ‘.’.
~	Appears at the beginning of every line in vi.

4.5 vi : Invoking

<u>COMMAND</u>	<u>DOES</u>
vi	Starts an empty edit buffer.
vi name	Edits the named file - creates named file if not exist
vi +3 name	Opens the named file and goes to third line.
vi + /bye name	Opens the named file and place the cursor at the first occurrence of bye

NOTE: The TERM variable to be set correctly and exported to the subsequent shells

Press i to enter insert mode from initial command (cmd) mode and to enter text.

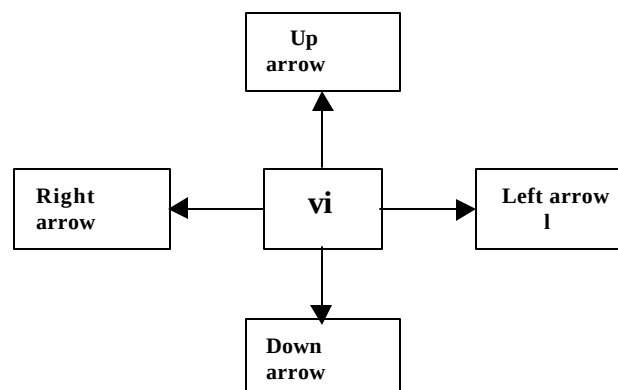
Press Esc repeatedly (F11 on Micro vax II!) till beep - this is command mode again.

Type ‘:wq ‘ to save and quit, from vi.

4.6 vi : Cursor movements (cmd mode commands)

Never to use arrow keys in vi for cursor movements (some terminals do support)

(Note that all vi commands involve the use of alphabetical keys, alone or in combination with Shift and Control Keys)



4.6.1 More cursor control in vi

<u>COMMAND</u>	<u>DOES</u>
#w	Moves forward by a word
#W	Moves forward by a word till next space
#e	Goes to last character in word
#E	Same as #e but ignores punctuation
f[character]	Up to a specified character in the current line - right
F[character]	Same as above but - left
t[character]	Same as f[c] but one c before
T[character]	Same as F[c] but one c before
#b	Go back a word
#B	Same as #b , ignores punctuation's
0 or ^	Go to beginning of a line
#\$	Go to the end of a line
#G	Go to (2G – go to 2nd line of file)
L	last line

Commands marked with # can be invoked with a number preceding them indicating repeat count

4.7 vi : Screen commands -Scrolling (cmd mode commands)

<u>COMMAND</u>	<u>DOES</u>
Ctrl-F	Scroll a screen forward - except last two lines
Ctrl-B	Scroll a screen backwards
Ctrl U	Scroll half a screen forwards
Ctrl-D	Scroll half a screen backwards
Ctrl R or Ctrl L	Re-draw and clear messages

Ctrl-G	Status of edit . Showing line no:, filename etc.
--------	--

4.8 vi : Deleting data (cmd mode commands)

<u>COMMAND</u>	<u>DOES</u>
#dd	Delete current line, where the cursor is present
#dw	Delete words from the cursor position
#dW	Same as #dw but including punctuation
#x	Delete character at the cursor position
#X	Delete a character before the cursor position
d\$ or D	Delete current line from the cursor position till the end of the line
d0	Delete current line from the cursor – up to start of line

4.9 vi : Text insertion

<u>COMMAND</u>	<u>DOES</u>
i	Invokes insert mode and inserts before character
I	Same as i but insertion at beginning of line
O	Opens blank line above current line
o	Same as O but below
a	Same as i but insert after cursor
A	Same as a but append at end of line
J	Join the line below with the current line

4.10 vi : Replacing existing text

<u>COMMAND</u>	<u>DOES</u>
r	Replace a single c at current cursor position
R	Invoke insert mode and go on replacing till Esc
s	Replace a single c with text - insert mode - continues till Esc - \$ appears after c to be replaced
S	Replace current line with new text - blanks entire current line - invokes insertion mode
cw	Replaces a word - \$ at end of word - replacement continues after end of \$ if word size greater than current word

C	Same as cw but acts on rest of line from current cursor position
cc	Same as C but entire line

NOTE : Flipping case : From command mode - cursor under character - Press ~ on keyboard.

4.11 vi : Cut and paste

<u>COMMAND</u>	<u>DOES</u>
#yy	Yank (cut) specified number of lines into yank buffer
p	Paste from yank buffer below current line
P	Same as p but above the current line

NOTE : dd also copies the lines to yank buffers. Imperative for paste to immediately follow cut.

4.12 vi : Saving and Exiting (colon : commands)

<u>COMMAND</u>	<u>DOES</u>
:q!	Quit and abandon changes
:wq	Save and quit
:x	Replace old copy with new and quit
ZZ	Save and Quit (from command mode)

4.13 vi : Block moves (colon : commands)

Copy 2nd line after 5th

:2co5

Copy multiple lines after the last line

:1,3co\$ **\$ for last line, 1,3 is range**

Move to line 6

:6 **cursor will move to line 6**

Delete three consecutive lines from the current position (#dd)

:3dd **alternative is :6,9d** (if the cursor is at line 6)

<u>Symbol</u>	<u>Means</u>
.	Current line where cursor
+	Line after current line

-	Line before current line
\$	Last line

Delete 3 lines below the current line

:.+3d

Delete 3 lines below current line including the current line

:.+3d **‘,’ denotes range**

Move line 1 to after line 3

:1mo3

Move lines 1,2 and 3 after line 5

:1,3mo5

4.14 vi : 9 lives

vi offers undo of up to 9 deletions (each deletion may be a group of lines) apart from the usual undo u . This follows the LIFO (Last In First Out) rule.

“#p or “#P - # is 1 to 9

4.15 vi : User defined registers

vi offers 26 registers (buffers) named ‘a’ to ‘z’ to the user into which text may be yanked and put anywhere else in the file using the ‘put’ commands. These assignments are active during the entire current session

<u>COMMAND</u>	<u>DOES</u>
“a5yy	Yank 5 whole lines including current and place in user defined buffer ‘a’
“ap	Put contents of ‘a’ below current line

All line, word and character commands can be used while yanking

4.16 vi : Marking and Yanking

<u>COMMAND</u>	<u>DOES</u>
mk	‘m’ for mark , ‘k’ is block name (In command mode)
‘k	Flips cursor to mark ‘k’ from anywhere else in file (In command mode)
d’k	Deletes block from mark ‘k’ up to current cursor position (In command mode)
“ay’k	Yanks into register ‘a’ (In command mode)
“ap	Puts contents of ‘a’ (In command mode)

4.17 vi : Pattern searching

<u>COMMAND</u>	<u>DOES</u>
/g	Search for pattern 'g' forwards
?g	Search for pattern 'g' backwards
n	Repeat the last / or ? command

4.18 vi : Substitute commands(Colon : commands)

To replace 'string' by 'string1' on the current line

:s/string/string1/ - replaces only first occurrence

:s/string/string1/g - replaces all occurrences

Substitution can occur on multiple lines if range is specified

:1,9s/string/string1/g

For substituting throughout the file

:1,\$s/string/string1/g

OR

:g/string/s//string1/g - note the double slash.

4.19 vi : Macros

vi macros are created using the 'abbr' command

:abbr uo "Unix operating system"

In insert mode press 'u o' and press Spc bar or Enter - the macro string is inserted at current cursor position.

:abbr -lists the macros defined

Macros can be unabbreviated or deleted

:una uo - deletes macro uo

4.20 vi : Mapping keys to commands

The 'map' command maps a user-defined key to a specific function

The cursor movement keys j,k,l,h may be mapped by default to their respective cursor movement keys

To map the key 'ctrl-X' to the 'wq' command

:map ^X :wq^M -note that ^M signifies Ctrl-M. In vi, ^ is typed by typing Ctrl-v

Note: All characters typed with 'ctrl' is taken as upper-case

An 'unmap' command is available to cancel assignments

:unmap ^X

Mapping in insert mode done using the 'map!' command

:map! ^x Sam fox - In insert mode 'Ctrl-x' will insert 'Sam fox' in the editor

:map displays only keys mapped to command mode functions

NOTE: An insert mode map cannot be unmapped unlike abbr

Note: To replace control-M present at the end of a line by a string

:1,\$s/^V// in vi editor

4.21 vi : Running shell commands

:sh temporarily 'shells out' to \$ and 'ctrl-d' returns the shell to vi

!<command> '!' stands for run which will directly run the <command>

4.22 vi : Reading command output

This is done using ':r!'

:r! date - reads in output of 'date' at current cursor position

:r! banner Hi there

4.23 vi : Editing multiple files simultaneously

vi can take multiple file arguments -

\$ vi file1 file2 OR

\$ vi f*

:args

[file1] file2 file3 - The bracketed file taken for display/edit

:n - Switches to next file in list

:args

file1 [file2] file3

:rew -rewind

:args

[file1] file2 file3 -i.e. 'rew' rewinds to file1

:rew! - discard changes and coercively rewind

:e file - edits file outside chosen list but :args will not show this file in list

4.24 vi : Read and Write across files

Write first five lines of text from file1 to file2-

:1,5 w! file2

Read same text from file2 to file1-

:1,5 r file2

4.25 vi : The set options

The 'set' option helps customize the vi environment -

: set all - displays all set options available

The more common options are:

<u>COMMAND</u>	<u>DEFAULT</u>	<u>DESCRIPTION</u>
1. :set number/nu	nonumber	Line numbers
2. :set showmode	noshowmode	Display vi mode
3. :set autowrite/aw	noaw	Save on n, rew or !
4. :set autoindent/ai	noai	Indent as first line
5. :set beautify/bf	nobeautify	Control characters omitted
6. :set directory/dir	dir-/tmp	Sets temporary directory
7. :set errorbells/eb	noeb	No error beep
8. :set ignorecase/ic	noic	Uppercase as lowercase
9. :set magic	magic	Allow all metacharacters
10. :set mesg	nomesg	Inhibit Write messages
11. :set redraw	noredraw	Act as intelligent terminal
12. :set scroll	scroll=half window	No. of lines to scroll
13. :set shell	sh=/bin/sh	Set which shell

14. :set term	value of TERM	Set which terminal
15. :set terse	noterse	Error messages terse
16. :set warn	warn	Warn save on Shell Esc
17. :set window dependent	window speed	No of lines in text window
18. :set wrapscan/ws	ws	Wrap pattern on find
19. :set warpmargin/wm	wm=0	Blank screen margin right
20. :set writeany/ wa	nowa	Allow writes freely

4.26 vi : The .exrc file

vi reads .exrc before loading a file and all options like map, abbr and set can be entered here so that they are permanent for that vi session. Here is an example of a typical .exrc file.

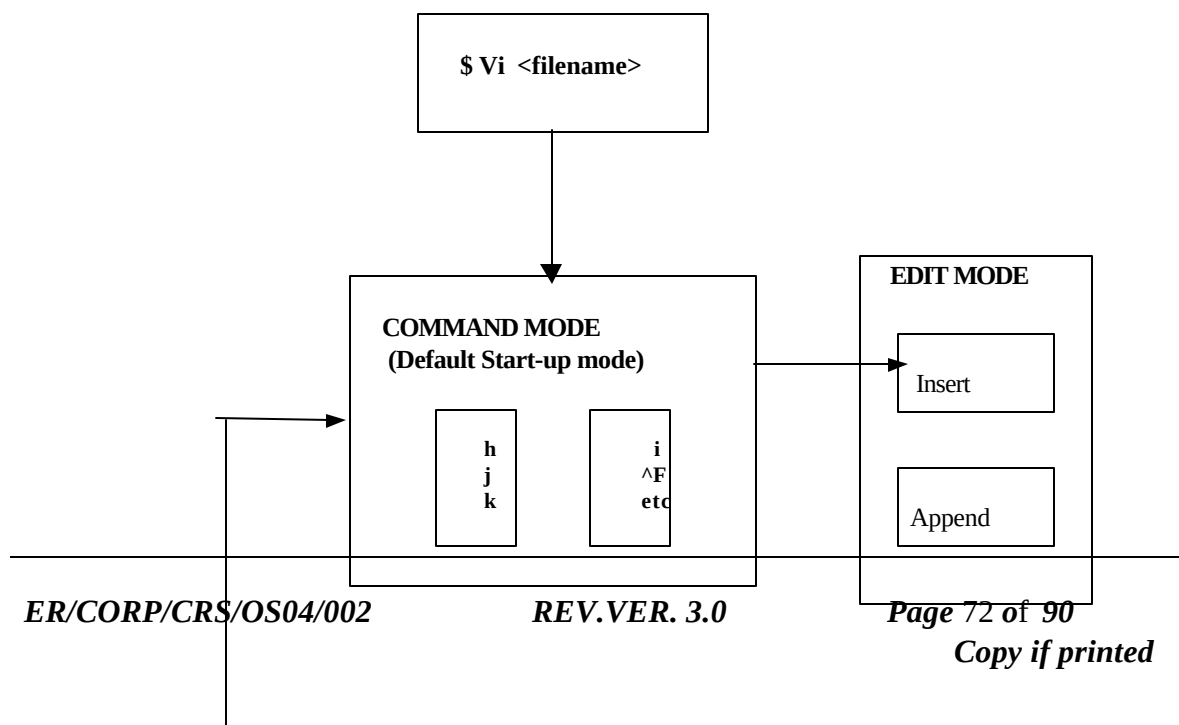
```
$ vi .exrc
```

```
set nu
```

```
set showmode
```

```
abbr uo Unix operating system
```

```
map ^k :r! date^M      note that ^M signifies Ctrl-M. In vi, ^ is typed by typing Ctrl-v
:wq
$
```

Vi 's Operating modes :

Assignment - 4

1. Find out the commands, which do not accept the repeat factors, in edit mode.
2. How does the significance of the repeat factor change when used with a \$?
3. Find out the difference between the following vi search commands-
/marketing and the corresponding ex command-
:/marketing/
4. What is the difference between the commands /* and f*?
5. Explain which of the following commands can be repeated or undone -
 - a) 40k
 - b) <control-f>
 - c) 5x
 - d) i main () <Esc>
6. Frame a command to delete the first two sentences of a line. How will you do that for subsequent lines?
7. How do you delete five lines above the current line, including the current line?
8. If you put a deleted sentence with p, does it put the sentence in the following line?
9. How will you interactively replace the string 'friend' with 'foe'? What is the equivalent instruction in the ex mode.
10. How will you copy all DO-WHILE structure in a C program into vi buffers.
11. Write a map, which interchanges the current line and the next one.
12. Write a mapping to transfer the contents of registers 'a' and 'b'.
13. Write a mapping to swap two adjacent characters.
14. Write a mapping for the key ^x which when pressed on a word displays the man page, if any, associated with that word. For example pressing ^x when the cursor is anywhere in the word 'who' should display the man page for who.

5 UNIX - SECURITY FEATURES

Keywords in this Chapter

<i>System security</i>	<i>Password</i>	<i>File security</i>
<i>Encryption</i>	<i>User-id</i>	<i>Su-id</i>
<i>Owner</i>	<i>Restricted Shell</i>	<i>File and directory permissions</i>
<i>Sticky bit</i>		

A multi-user / networked environment holds great potential for the professional hacker. In order to preserve data confidentiality, security becomes a key issue. To meet this challenge, Unix has set several security standards. Unix makes users aware of their security rights, privileges and prerogatives and helps the user to build his own fort.

5.1 Password - the shield

To access (log on to) a Unix system a user needs to have:

1. A valid login name
2. A password

The /etc/passwd file contains the database containing valid user passwords (passwords encrypted).

The fields in the passwd file are:

login-id:encrypted-password:uid:group-id:miscellany:login-directory:shell

1. login-id : the login name of the user
2. encrypted password : user password in encrypted form or the existence of the password is represented by * / X .
3. uid : a unique identification number for the user
4. group-id : a unique identification number for the group to which a user belongs
5. miscellany : a brief comment on the user
6. login-directory : path of HOME
7. shell : a shell that is active at login

5.2 Good passwords

The characteristics of a good password are:

1. To be a heterogeneous mix of characters and metacharacters, difficult to guess
2. To be easy on the user's memory- no writing down
3. To be changed frequently – at least once in every 6 months

4. To be highly secretive - never used before or in use elsewhere

A good password should

1. Never have any common proper names
2. Not be connected with anyone - telephone numbers etc.,
3. Not be lent to anyone

5.3 Changing passwords

No one but the actual user can change his password.

\$ passwd

Setting password for user: user1

Old password:

New password:

Re-enter password:

\$

Passwords are likely to be rejected by the system if

1. The length of the password is less than as set by the PASSLENGTH variable
2. The new password is same as old password
3. The password and login name are the same
4. The password is some obvious combination of the login name

Passwords have an expiry date associated with them that can be set by the user. The system administrator or SU can 'lock-out' a user for security breaches

5.4 File security

Unix offers read (r), write (w) and execute (x) permissions that can be set for each file by the user. The weights associated with each of these attributes are r=4,w=2 and x=1. These are used in assigning multiple permissions to the files.

The r,w,x permissions for each file can be set for three distinct classes of users:

1. The owner of the file
2. Those belonging to the same group as the owner
3. All others

Therefore, a nine-bit field decides the composite permission on each file.

5.4.1 Assigning file permissions

The following table brings out the method of calculating file permissions:

Owner	Group	Others
<i>r w x</i>	<i>r w x</i>	<i>r w x</i>
4	4	4
2	2	2
1	1	1
<i>Digit</i>	<i>Digit</i>	<i>Digit</i>

Suppose we want to set permission **rw-r-x-r-x** on a <file> then the table would be

Owner	Group	Others
<i>r w x</i>	<i>r w x</i>	<i>r w x</i>
4	4	4
2		
1	1	1
7	5	5

\$ **chmod 755 <file>** - Sets the above permissions on <file>

5.4.2 Default permissions for file creation

System wide default permission for a newly created file is rw-rw-rw which works out to 666.

The system variable UMASK (User file creation mask) sets this default.

\$ **umask**

022 ———— *Note:* This number refers to permissions denied to the three groups.

Here, no permission is denied to owner (0), write permission denied to group and others (2 2).

So, to calculate the system default file permissions pertaining to above UMASK setting:

6 6 6	System wide default permissions
0 2 2	Denial 'mask' set by UMASK
6 4 4	Resultant permissions that will be set on all files created (rw-r--r--)

The system wide default permissions for a directory is 777 i.e., rwxrwxrwx.

Note: No execute (x) permission on a directory means you can't enter (cd) into it.

The UMASK variable also determines the resultant permission for a directory. On similar lines, the value here would be

7 7 7	———	System wide default permissions
0 2 2	———	Denial 'mask' set by UMASK
7 5 5	———	Resultant permissions that will be set on all files created (drwxr-xr-x)

5.4.3 Altering file permissions

It is possible to change file permissions symbolically without having to calculate octal permission numbers as above. The chmod command, once again, can be used as follows for this purpose:

chmod [who] [+ - =] [permission] <file>

<i>who</i>	<i>+/-/=</i>	<i>permission</i>
<i>u - user or owner</i>	<i>+</i>	<i>adds r,w or x</i>
<i>g - group</i>	<i>-</i>	<i>removes</i>
<i>o - Others</i>	<i>=</i>	<i>assigns specified permissions removing others if any.</i>

Example

Remove r and w permissions for 'Others' from file 'textfile'

\$ chmod o -rw textfile

Note that a blank first argument here denotes all users.

\$ chmod +x textfile **- Makes textfile executable for all users**

5.4.4 Changing owners and groups of files

To change the owner of a file the chown command is used:

\$ chown user2 textfile **- Makes user2 the owner of textfile**

To change the group to which a file belongs, the chgrp command is used.

\$ chgrp newgroup textfile **- Assigns text file to group 'newgroup'**

5.5 Directory permissions in Unix

Since the directory is also a file in Unix, a directory too has the r, w and x permissions associated with it. However, the meanings of these permissions for a directory are not the same as they are for an ordinary file. Several cases are examined:

5.5.1 'r' permission only

Can only list the contents of the directory, nothing else is possible.

5.5.2 'w' permission only

No listing files, no cd, nothing can be done on that directory!

5.5.3 'x' permission only

Can enter into the directory, but nothing can be done on that directory

5.5.4 'w' and 'x' permissions only

Everything is possible, except listing of files.

5.5.5 'r' and 'x' permissions only

Can't create a new file, can edit or delete an existing one either. It makes sense to deny write permission on a directory to others.

5.5.6 'r' and 'w' permission only

Nothing is possible because can't enter into the directory, same as giving write permission alone.

So, the ideal directory permission would be:

drwxr-xr-- (Value 754 octal)

- Full powers for the owner.
- No modification or creation of files by group, only execution and listing.
- Others get just listing of directory only.

5.6 The sticky bit - ultimate directory protection

When the sticky bit for a directory is set it means that there can be no deletion or overwriting of existing files in the directory. Existing files may be read and new ones created if and only if the usual permissions permit it. Therefore, existing files in the directory cannot be harmed under any condition when the sticky bit is set.

\$ chmod u+t crucial

sets the sticky bit for directory 'crucial'

So, the ideal permission for a directory will now be:

drwxr-xr-t

5.7 The su command

The su (Substitute User-id) command switches a user to 'Super User' without having to logout and is equivalent to logging in as root. Of course it requires that the user should know the password of the root.

It is also possible to switch login as any user provided that user's password is known.

```
$ su loginname
```

5.8 The su-id bit - a temporary privilege

If the su-id bit is set for an executable file, whoever executes that file, the file is assigned the group-id of the owner for that executable file. This enables the user to temporarily assume the permissions of the file owner's group and therefore he could edit a file of that group. On returning to the shell the original group-id is restored.

Example

```
$ chmod u+s data-entry
```

Sets the 'su-id' bit for executable file 'data-entry'

Now, any user who invokes data-entry will automatically be allowed to edit any file belonging to the group to which data-entry belongs. This method is used to edit the passwd file.

```
$ ls -l /etc/passwd /bin/passwd
```

Database file (protected) Executable

```
lrwxrwxrwx 1 root root 34 Oct 20 16:51 /bin/passwd -> /opt/K/SCO/Unix/5.0.0Cl/bin/passwd
```

```
-rw-rw-r-- 1 bin auth 1893 Jan 28 10:45 /etc/passwd
```

```
$ ls -l /opt/K/SCO/Unix/5.0.0Cl/bin/passwd
```

```
----x--s--x 1 bin auth 57852 Apr 23 1995 /opt/K/SCO/Unix/5.0.0Cl/bin/passwd
```

Note: The database passwd cannot be edited normally but any user who invokes the executable passwd can modify the database. This adds extra safety to the file passwd.

5.9 The restricted shell

The restricted shell is used by the SU to impose restricted access on naive, amateur and temporary users. If the restricted shell is attached to a user then -

1. Access is clamped to HOME directory alone.
2. No change in PATH variable is allowed.
3. No new files can be created or existing ones can be modified.

Note: If rsh is specified as a sub-shell then it can easily be killed by using the 'kill' command with the PID drawn from the 'ps' command.

5.10 File encryption

The 'crypt' command uses a 'key' to encrypt its standard input. It also performs the converse if the correct 'key' is specified. After encryption the original file is to be destroyed. While encryption non-trivial keys are to be used to ensure foolproof protection.

Assignment - 5

1. *How is /etc/passwd updated by any user, while changing password, even though the file doesn't have the write permission.*
2. *Why is the complete pathname specified in the last field in /etc/passwd ?*
3. *Why is the crontab file a likely candidate for malicious alteration by a system penetrator ?*
4. *What is the main difference between the last and lastcomm commands?*
5. *Which file does the last command examine? Name another command, which can be used to examine the same file.*
6. *Why is a file such as a Unix editor (that has the su-id permission for root and execute permission set for group and other users) has a potential security problem?*
7. *Outline the method of calculating system wide defaults for file creation using UMASK.*

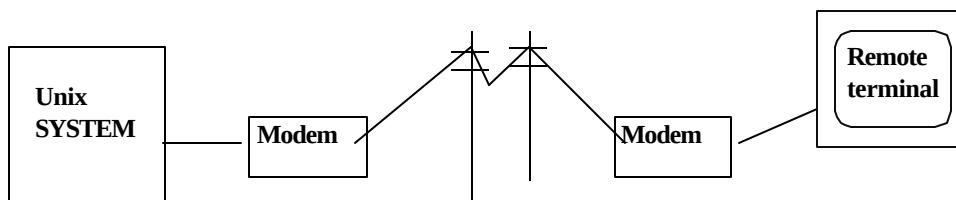
6 ELEMENTARY UNIX - UNIX COMMUNICATIONS

Keywords in this Chapter

<i>Dial-up communication</i>	<i>Network Manager</i>	<i>Networking</i>
<i>Remote Terminal</i>	<i>Network Topology</i>	<i>Modem</i>

6.1 Telephone communications

The physical set-up of Unix System is in such way that it can be accessed from *remote terminal* through telephone lines driven by *modems*. This is known as a *dial-up* connection.



The bottleneck here is that *modems* are slow. The maximum throughput can be up to 19,200 baud in direct connections whereas modems are typically 2400/4800 baud. Such systems usually have *dial-out* capability i.e. they can make as well as receive calls.

6.2 Modern communication methods

6.2.1 Inter-system 'mail'

The 'mail' command can be used to communicate among users on the same (single) system or with the users on the other systems.

To send a mail to a user john on the same system:

```
$ mail john
```

6.2.2 mesg

The *mesg* command allows you to assign permissions to your terminal for receiving messages. The *mesg* command with argument *n* forbids messages via write(C). *mesg* with argument *y* reinstates permission. By itself, *mesg* reports the current state without changing it.

```
$ mesg
mesg is n
```

6.2.3 write

The *write* command allows you to write to another user's terminal. *write* command cannot communicate with users who are not currently logged on to the system. If the user is logged in more than once, the *tty* argument may be used to indicate the appropriate terminal

```
$ write userid [ tty ]
Hi .....
...
ctrl-D
```

When first called, it sends the message:

```
Message from logname tty ...
Hi .....
..
(end of message)
```

6.2.4 wall

The *wall* command writes the message to all users terminals. *wall* sends the message to all the users currently logged in. It was designed for the use by system administrator to send messages to all users. The *wall* command used by system administrator overrides all write permission constraints.

```
$ /etc/wall [ -g group ] [ file ]
-g      sends message only to that group
```

The *wall* command reads a message from the file (if a file is specified) or the standard input until an end-of-file is received., and send this message to all users currently logged in.

6.2.5 ftp

The *ftp* program allows you to transfer files to and from a remote computer and work with files and directories on that remote machine.

```
$ ftp [options] [host name]
```

Host name can be a domain address or an IP address.

Once the connection is complete, ftp enters command mode.

<u>FILE TRANSFER COMMANDS</u>	<u>DOES</u>
-------------------------------	-------------

get [remote-filename [local-filename]]	Downloads the file from the remote machine specified by remote-filename and renames the downloaded file to local-filename on the local machine
put[local-filename [remote-filename]]	Uploads the file specified by local-filename to the remote computer and renames the uploaded file to remote-filename
quit	Close the connection to the remote computer and terminates ftp

6.2.6 telnet

The *telnet* command allows the user to manually log into a remote system using the TELNET protocol.

Unlike *ftp*, *telnet* connection is a general purpose connection.

The syntax is

\$ telnet [address [port]]

where the address may be a domain address or an IP address, and the port specifies the port number on the remote computer you want to log into.

telnet can be used to perform any operation that could normally be done using a terminal connected directly to the remote computer.

Assignment - 6

1. *Explain what you understand by the term 'dial-up' communication ? What is its chief disadvantage ?*
2. *Explain the method (with notations) used for forwarding inter-system mail ? (mail command)*
3. *What are the chief advantages & disadvantages of UUCP ?*
4. *Explain what exactly the command cu does ? What is this mode called and why ?*
5. *Explain the need for Networking Unix systems.*

7 UNIX - PROGRAM DEVELOPMENT

Keywords in this chapter

<i>adb</i>	<i>cc</i>	<i>make</i>
<i>SCCS</i>	<i>dependencies</i>	<i>admin</i>
<i>delta</i>	<i>prs</i>	

One of the primary goals of the Unix operating system design was ease of application program development. As such the Unix system offers a wide assortment of programming languages and development tools. Some of the programming languages available on a typical Unix system are :-

Program	Primarily used for
awk	String processing, report generation, file editing
C	General purpose and system programs; fast executing programs
efl	Writing structured FORTRAN programs
Fortran 77	Number-crunching; engineering, statistical and mathematical applications; fast executing programs
sh	General purpose programs; file manipulation; process control
sno (SNOBOL)	Pattern matching; string manipulation

The nature of the application would determine the choice of a language. In most cases there is a trade-off between development time versus execution speed. It is not uncommon for the shell itself to either control the execution of other programs, or do a lot of processing itself. A shell or a awk script is shorter and takes less time to develop than an equivalent C program but runs more slowly because it is interpreted rather than compiled. This chapter discusses about the tools available for the C program development .

7.1 C PROGRAM DEVELOPMENT

7.1.1 cc - C language compiler

A C program has to be ended with **.c** extension to compile in Unix .

The **cc** command is used to compile a C program -

cc files

where *files* is a list of files to be compiled.

If no-error is detected by the compiler, an object file would be created with the same name as the source file and the **.c**

extension being replaced by .o

The final phase of compilation involves *linking (or loading)* the object file to create an *executable object*. In this phase, the compiler tries to resolve external references against the standard C library **libc**. This library contains the *standard I/O library* routines such as *printf* and *fopen*, as well as other functions such as *strcat*, *malloc*, and so on.

If all external references are satisfied, then an executable object file called **a.out** will be created. To run this it may be invoked as any other Unix command.

\$ a.out <Enter>

Note: If there is only one .c file for compilation, then the corresponding .o file is deleted after compilation. However, if more than one .c file is used during compilation, the corresponding .o files will be retained.

Where more than one file is specified, the **cc** command can take .c as well as .o files as arguments

\$ cc main.o init.c process.o cleanup.o

Note that here the only changed file is init.c while the .o files are from a previous compilation.

7.1.1.1 Important 'cc' options

1. The -c option specifies that no linking is to be done and forces just the creation of the .o file.
2. The -O option causes a special optimization program to be run to improve the efficiency of the object code.
3. The -o option lets a different name to be specified for the final executable rather than the default a.out
4. The -l options are particularly useful when using libraries other than the standard C library-
5. The -lm (place after file names on cc) specifies the Unix math library.

7.2 C PROGRAMMING TOOLS

Tool	Used for
adb	Debugging C programs (better to use sdb or ctrace if available)
cb	Automatic formatting of C programs
cflow	Generating a flow graph of external references
ctrace	Tracing execution of a C program statement by statement
cxref	Generating a cross-reference listing of a C program
lint	Checking a C program (that may span many files) for bugs or non-portable uses of the language. (For checking unused variable & function whose return values not used etc. can be detected. Some casting related problems can be detected.)
make	Regenerating a program system by automatically tracking files that have changed since the system was last made.
prof	Obtaining performance data such as the number of times a function is called and the amount of time spent in each function
SCCS	Maintaining and updating source programs; automatic version tracking

CSCOPE	For analysing, re-engineering of the existing complex code
sdb	Debugging C programs; symbolic debugger

7.2.1 THE MAKE PROGRAM

make is particularly useful in the program system that comprise of more than one source file. It automatically keeps track of the source files that have been edited (changed) since the last compilation and causes their recompilation when necessary. It also automatically re-links the object modules to update the final executable.

The **make** program takes a file, known as the *makefile*, as its input. This file describes the following to the **make** command-

- The names of the source files that make up the program system
- Their interdependencies
- How to regenerate the program system

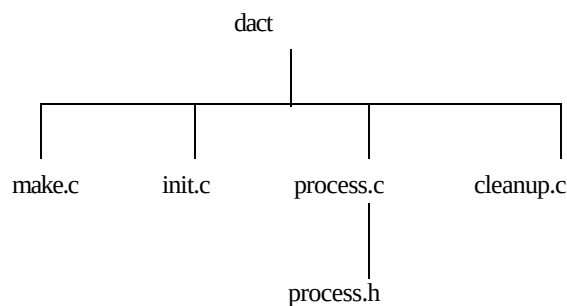
The make command will automatically scan the *makefile* or *Makefile* on invocation. However, the makefile can be renamed in which case the name will have to be supplied to make command as an argument.

\$ make -f filename

Example

Assume that a program called dact is contained in four C source files called main.c, init.c, process.c and cleanup.c. Also suppose that the file process.c needs a file of definitions called process.h.

The file dependencies can be depicted as -



The make script will be as follows-

\$ cat makefile

makefile to create the dact system -

dact : main.o init.o process.o cleanup.o

< Tab > cc -o dact main.o init.o process.o cleanup.o

main.o : main.c

Since 'make' knows that a .o file depends on


```

init.o : init.c ← its corresponding .c file these lines may
process.o : process.c process.h ← be omitted.
cleanup.o : cleanup.c
$

```

Now to generate an up to date version of dact -

```
$ make dact
```

make now automatically looks into the makefile/Makefile and proceeds to compile dact conditionally such that only the changed files participate in the compilation process.

7.2.2 THE SOURCE CODE CONTROL SYSTEM (SCCS)

A package of programs called SCCS is available to help and manage the development and maintenance of large programs. The main purpose of the SCCS system is to automatically track changes between versions of a program. The SCCS system maintains a list of these changes and has the ability to quickly recreate a particular version on request.

Since SCCS only keeps track of the changes between versions and not the actual complete versions themselves, it does not waste much disk space.

A program system that has already been developed without using SCCS can easily be placed under the SCCS system

7.2.2.1 Placing a program under SCCS

The basic format of the **admin** command that is used to place a system of programs under SCCS is :

```
admin -n -ifile s.sccsname
```

This command will place *file* under SCCS, giving it the name *s.sccsname*. *sccsname* and *file* are one and the same. Note that *all* SCCS files must begin with s.

The admin command must be run separately for each file that is to be placed under SCCS.

Example:

```
$ admin -n -i main.c s.main.c
```

places **main.c** under SCCS, calling it **s.main.c**

In order to place all .c files under SCCS it is best to use the *for* statement in a *shell script* -

```

for f in *.c
do
admin -n -i $f s.$f
done

```

Every SCCS file has a number associated with it known as its *version number*. This number starts at 1.1, which means *release 1, level 1*. As and when a change is made to an SCCS file, 1 gets automatically added to its *level number*.

7.2.2.2 Extracting files from SCCS

Once source files have been placed under SCCS, the original **.c** files are to be removed leaving only the **s.** equivalents. Henceforth any access to source files must be through the SCCS system only, including any display of files.

The *get* command is used for extracting from SCCS source files for different purposes. It takes several options of which the important ones are—

To display the latest version of source in *s.main.c* on the terminal -

```
$ get -p s.main.c
```

To extract *s.main.c* and place it in *main.c* which may then be taken for editing -

```
$ get -e s.main.c
```

To extract but not edit (may be just for compilation) -

```
$ get s.main.c
```

Note: 'get' also supports an option that allows the user to extract a specific version of a file. Thus, a specific version of all files belonging to a project may be extracted and 'remade' to obtain the desired executable (using the 'make' utility).

7.2.2.3 Recording editing changes

Once the file extracted from SCCS has been edited and saved it is important to restore it under SCCS. This is done with the **delta** command. It also automatically updates the current level number of that file and prompts the user for an optional comment to be stored with this version of the file. The delta program then removes the file that was edited.

```
$ delta s.main.c
```

Record changes

```
comments? Fix timing bug
```

Comments are entered

```

Ni id keywords (cm7)

```

```
1.2
```

```
1 inserted
```

```
0 deleted
```

```
4 changed
```

```
$
```

→ New version number and summary of changes displayed.

7.2.2.4 Information on SCCS files

The **prs** command is used to obtain information about SCCS files - without any options *prs* will display a revision history -

```
$ prs s.main.c
```

```
s.main.c:
```

```
D 1.2 84/04/15 17:59:21 steve 2 100001/00000/00004
```

```
MRs:
```

COMMENTS:

fix timing bug

D 1.1 84/04/15 17:58:36 steve 1 000004/00000/00000

MRs:

COMMENTS :

date and time created 84/04/15 17:58:36 by steve

The four basic SCCS commands are :

Command	Used to
admin	creates new SCCS files
delta	make change to a SCCS file
get	extract SCCS files
prs	print information about an SCCS file

***Note :** The SCCS and ‘make’ utilities work well in conjunction with each other. If make doesn’t find a particular file **f** it will automatically look for file **s.f**. If it finds it, it will automatically assume it as an SCCS file and will issue the necessary **get** command to extract it. After **make** is finished working with the extracted SCCS file, it will automatically remove it.*

7.2.2.5 mdiff

mdiff command on SCCS shows the differences between a local file and the corresponding SCCS reference file.

Example:

```
#!/bin/ksh
#
# mdiff: shows the differences between a local file and the corresponding
#      SCCS reference file.
#
# HPR - dd mmm yyyy
if [ $1 = "-w" ]
then
    options=$1
    shift
fi

if [ $1 = "-i" ]
then
    options="$${options} $1"
```

```

        shift
    fi

    for i in $*
    do
        bi=$(basename $i)
        if [ -f "$xxxx_DDL/$bi" ]; then sfile="$xxxx_DDL/$bi";
        elif [ -f "$xxxx_SQL/$bi" ]; then sfile="$xxxx_SQL/$bi";
        elif [ -f "$xxxx_SCRIPTS/$bi" ]; then sfile="$xxxx_SCRIPTS/$bi";
        elif [ -f "$xxxx_STATIC_DATA/$bi" ]; then sfile="$xxxx_STATIC_DATA/$bi";
        elif [ -f "$xxxx_SRC/$bi" ]; then sfile="$xxxx_SRC/$bi";
        else
            echo "$i: not in SCCS"
            continue
        fi

        echo $i:
        echo =====
        eval diff $options $i $sfile
    done
    exit 0

```

7.2.2.6 whit

What Have I Taken (from SCCS)

Usage: whit [-c] [-b] [username]

Example:

```

u_count=0
for username in $userlist
do
    echo "$username:"
    for pfile in $(grep -slw -- "$username" $plist)
    do
        if [ "$no_detail" -eq 0 ]
        then
            echo `cat $pfile`: $(echo $pfile | cut -f2- -d'.')
        else
            echo $(echo $pfile | cut -f2- -d'.')
        fi
    done | sed 's/^/      /' | eval "$count_cmd"
    echo
    u_count=$(expr $u_count + 1)
done

if [ "$county_only" -eq 1 -a "$u_count" -gt 1 ]

```

```

then
    echo "Total (for all users)": $(find $xxxx_HOME/sccs -name "p.*" | wc -l)
fi

exit 0

```

7.2.2.7 Tar

transfer_files.ksh:

This script creates a tar of all the files under some directory(SCCS_MIRROR) which have changed since last transfer

```

#!/bin/ksh
# *****
# Application : XX
# File       : transfer_files.ksh
# Description : This script creates a tar of all the files under SCCS_MIRROR
#              which have changed since last transfer
#
# Files Used :
#             $XXX_HOME/sccs_mirror -- takes all files under this dir
#             $XXX_HOME/logs/transfer_files.log -- log of file transfer
#             $XXX_HOME/logs/last_transfer.dat -- timestamp of last transfer
#
# Usage      : transfer_files.ksh
#
# Modification History:
#
# Date      Person      Remarks
# -----
# dd-mmm-yy Ramesh/Sanjeev Initial version.
# *****/
#

. $XXX_SCRIPTS/XX_shell_lib.ksh
timestamp_file=$XXX_HOME/transfer/last_transfer.dat
tar_filename=$XXX_HOME/transfer/transfer_$(date +%y%m%d%H%M%S").tar
transfer_logfile=$XXX_HOME/transfer/transfer.log
progrname=$(basename "$0")

psl_mktmp logfile prefix=log suffix=log

psl_start "$0"

psl_env_check XXX_HOME XXX_SCCS_MIRROR
check_file_exists -d \
    "$XXX_HOME" \
    "$XXX_HOME/transfer" \
    "$XXX_SCCS_MIRROR"

```

```
check_write_access "$progname" "$star_filename" "$transfer_logfile"

if [ ! -f "$timestamp_file" ]
then
    echo "$(date): Last tranfer data not available; doing FULL transfer" | tee -a "$transfer_logfile"
    cat > "$timestamp_file" << -EOF_TIMESTAMP
$(date): Full transfer
-EOF_TIMESTAMP
    exit_if_error $? "Couldn't create time stamp file '$timestamp_file'"

    #
    # set timestamp to a old date
    #
    touch -t 197001010930 "$timestamp_file"
    exit_if_error $? "Couldn't touch time stamp file '$timestamp_file'"
fi

OLDDIR=$PWD
cd "$XXX_SCCS_MIRROR"
echo "===== " >> "$transfer_logfile"
echo "$(date): Transferring files" >> "$transfer_logfile"
file_list=$(find . -newer "$timestamp_file" -a -type f -print)
if [ ! -z "$file_list" ]
then
    tar cvf "$star_filename" $file_list 2>&1 | tee "$logfile" >> "$transfer_logfile"
    exit_if_error $? "Unable to create tar file '$star_filename'"

    cat "$logfile"
    rm -f "$logfile"

    echo "$(date): Incremental transfer" >> "$timestamp_file"
    exit_if_error $? "Unable to write to time stamp file '$timestamp_file'"
else
    echo "$(date): No files to transfer"
    echo "$(date): No files to transfer" >> "$transfer_logfile"
fi

psl_end S
```

Assignment - 7

1. *Write a simple C program. Use make to compile and link it. Use make again. Then use touch on the source. Use make again. Explain the results.*
2. *Choose any C application and write the dependency diagram for it. Write the makefile (make script) and use the same to compile and link the application.*
3. *What is the purpose of SCCS ? Outline the method used to launch a C application into the control of SCCS ? Describe the SCCS commands used in the process.*
4. *SCCS and make are made for each other. Comment with example.*
5. *Write notes on the different Programming tools available in the Unix environment that you are aware of illustrating the concepts with suitable schematic diagrams.*

8 An introduction to interprocess communication

Keywords in this chapter

<i>Pipes</i>	<i>Shared memory</i>	
<i>Named pipe</i>	<i>Message Queues</i>	
<i>FIFO</i>		

Many a time processes need to communicate with each other. One typical scenario is one involving data transfer from one process to another. Unix operating system provides several mechanisms for inter-process communication. These include

1. Pipes;
2. FIFOs;
3. Message queues; and
4. Shared memory.

Pipes are used for communication between a parent and a child. A pipe involves the opening of two file descriptors, one for reading, the other for writing. Typically, the parent process uses the second descriptor to write data and the child process uses the first descriptor to read data.

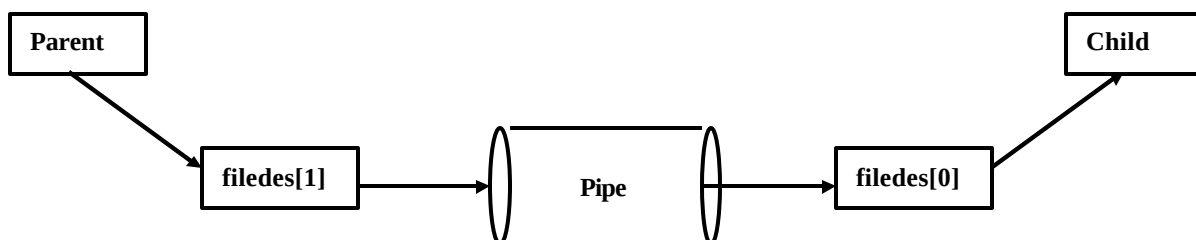
The system call `pipe` can be used to establish a pipe. It has the following form:

`int pipe(int fildes[2]);`

The system call returns two file descriptors, `fildes[0]` for reading and `fildes[1]` for writing.

Unidirectional pipes

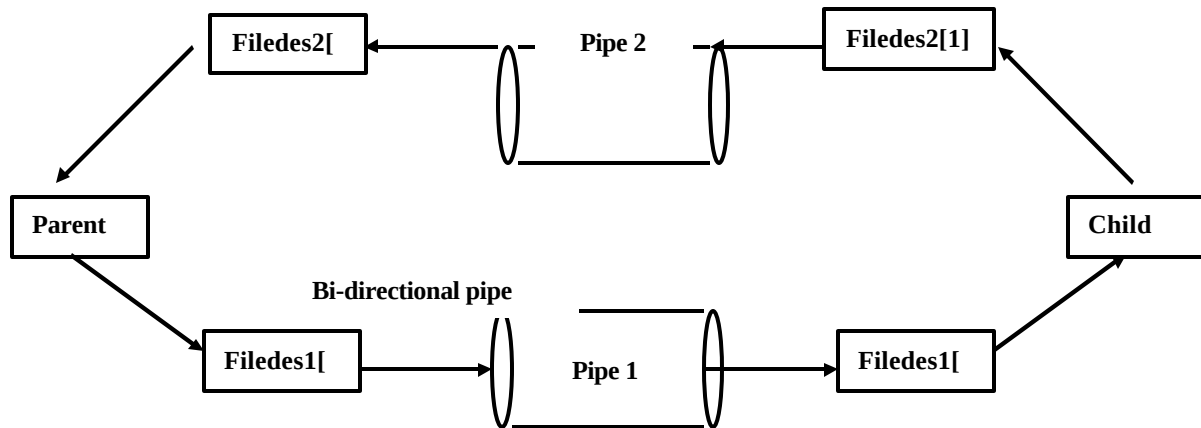
In a unidirectional pipe the parent closes the read file descriptor and the child closes the write file descriptor. This would result the data flow in a single direction as shown in the figure.



Unidirectional pipe

Bi-directional pipe

In some applications parent and child processes need to talk to each other i.e. there will be data flow in both directions. In such situations a bi-directional pipe is used, which is implemented using two pipes. The first pipe is used for data flow from parent to child and the second pipe for the reverse direction. This is illustrated in the following diagram.



Named Pipes

A pipe can be used only between related processes. This is because the file descriptors that form the pipe have to be shared between two processes. This is easy in a parent-child relationship since that permits inheritance of file descriptors. One of the mechanisms that permit communication between unrelated processes is a name pipe or FIFO (First In First Out).

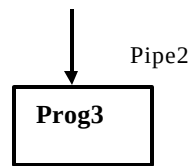
What is a FIFO?

- It is a file with a filename and type as pipe.
- Several processes could write into it one after the other.
- Unlike a file, as soon as a process reads from it that data is emptied from the FIFO

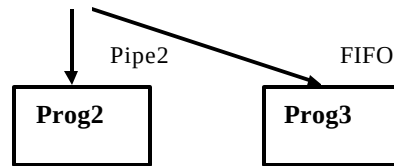
What a FIFO can do and What a pipe cannot?

a) A pipe is used to transfer data when processes are connected serially (data flows from one process to another sequentially). Since FIFOs have names they can be used to send the same data to multiple processes which are not connected serially. The *tee* program can be used to duplicate output data and send it to a file. The following sequence of steps illustrate this.





Serial data flow through pipes



Parallel data flow using FIFO

1. The following command creates a FIFO called f1

```
mkfifo f1
```

2. The following command redirects output from who to word count as well as to f1

```
who | tee f1 | grep -i benny &
```

3. The following command searches for a string benny in f1

```
wc -l < f1; echo "(Total number of users)";
```

4. The following command removes the FIFO

```
rm f1
```

b) Multiple unrelated processes can communicate to each other using FIFOs. When processes that are independently created require to communicate with each other, named pipes (FIFOs) have to be used. This is because there is no means to transfer open file descriptors across unrelated processes. This is common in client-server applications. Clients and servers exist as unrelated processes. They transfer data through FIFOs whose names have been agreed upon by some convention.

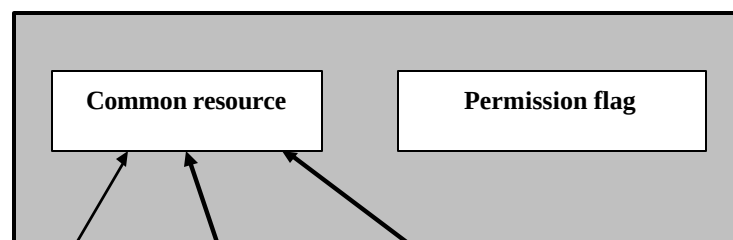
Semaphores

Many a time unrelated processes need to access and modify common resources (for example, files). When a process accesses certain data it has to make sure that some other process is not modifying it at the same time. Record locking is one means of exercising control over shared resources. One major disadvantage with record locking is, a process does not know when to retry after an attempt to lock a file has failed (due to an existing lock).

There are two features required from the system kernel to control shared resources.

1. There should be a means of informing all other processes that a certain process is accessing a common resource (something like a variable that is global across different processes).

2. The system should automatically inform waiting processes when a resource has become free (instead of the process trying again and again by itself).





A semaphore provides the above functionality. It can be seen as a global permission flag (independent of individual processes) for accessing a common resource. When a process modifies a common data, it turns the permission flag off to other processes. It turns the flag on after it has done with the resource. The operating system keeps other processes waiting till the permission flag has been set so that they need not keep checking the flag at regular intervals.

In the System V implementation a semaphore is associated with a set of values. A number of operations can be done on this set of values automatically. The following operations are defined on a semaphore value.

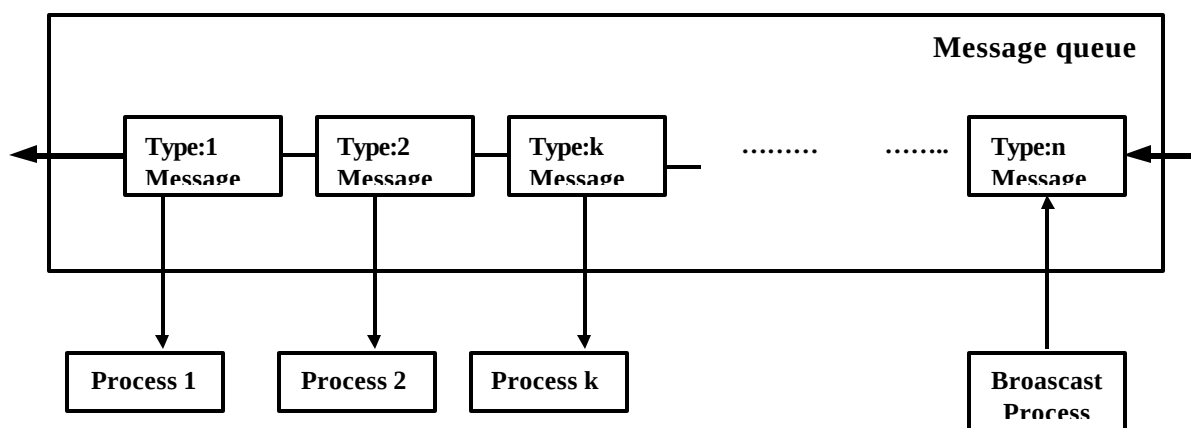
1. Increment
2. Decrement.

The increment operation increases a semaphore value by a specified amount. The decrement operation goes through only if the result of decrement would result in a number greater than or equal to zero. For example, if the semaphore value is already zero and a process has requested a decrement by one, the system will make the process wait till some other process increments the semaphore value. This property can be used to synchronize processes. In addition to increment and decrement operations, a process can also make a request to wait till the semaphore value becomes zero.

Message queues

Message queues can be used by unrelated processes for inter-process communication. The following are the typical operations involved in IPC using message queues.

1. A message queue is created with a unique identifier.
2. A set of processes send messages to the queue.
3. Another set of processes receive these messages.



Any process that has access permission for a particular message queue could receive any message in the queue.

However, in practice a message type is specified when a message is sent, and specific processes read only specific type of messages.

Two basic operations are sending a message and receiving a message. The messages that are sent goes into the queue and are removed when processes read them.

Shared Memory

Shared memory is another means of IPC. Different processes share a segment of memory. These processes can write data to this memory to be used be read by other processes and so on. Exchange of data in this manner is very fast.

The operations involved in the use of shared memory are summarized below:

1. A shared memory identifier `shmid` is obtained using the call `shmget`.
2. The pointer to the location where the shared memory is attached is obtained using the call `shmat`.
3. Data is stored (or retrieved) in the given location. A simple means of doing it is using `memcpy` function.
4. Shared memory could be detached using the call `shmdt`.

REFERENCES

1. **Bach, M: *The Design of the Unix Operating System* . Prentice-Hall, 1986**
This book is the de-facto standard for Unix internals. The book is colossal explaining in detail the internal algorithms and data structures in Unix right from the file system up to interposes communications and distributed Unix Systems. The treatment is highly logical and the schematics used are excellent
2. **J.J. Manger: *UNIX - The Complete Book, A guide for the professional users*. Galgotia ,1991**
This is a relatively new book on Unix and as such focuses on many modern Unix issues like Networking, X window system etc. The chapter on communications is exhaustive. This book is a good gateway for getting into higher Unix.
3. **R. Groff, P. N. Weinberg: *Understanding Unix - A conceptual guide* . BPB .**
This is the book for a bird's eye view of Unix. Compares Unix to other OS's and discusses all its flavors. The book is profusely illustrated with some top class schematics and diagrams. The book is predominantly 'features' oriented.
4. **Kernighan Brain W and Pike Rob: *The UNIX Programming Environment*. Prentice-Hall, 1984**
This book is the de-facto standard for studying Unix from the User's perspective. The book is sprinkled with numerous illustrative examples of a very high order especially in chapters on Shell Programming, Awk and Program Development. An algorithmic approach to problems is another highlight. Emphasizes the Unix philosophy throughout. Read from cover to cover - a must.
5. **Kochan Stephen G. and Patrick H Wood: *Exploring the Unix system* . Hayden , 1989**
This is a well written book emphasizing concepts. It covers a lot of ground including Communication and Administration. The highlight of this book is its simplicity and therefore suitable for a cover-to cover first read.
6. **Prata Stephen: *Advanced UNIX - A Programmer's Guide*. Hayden , 1990**
This book from the Waite Group is another indispensable book for Unix users. The treatment is in-depth and an high-point is the many schematics that make concepts clearer. Many problems are solved after thorough analysis. This is the book to refer for Shell Programming and Awk.
7. **Sumitabha Das: *UNIX system V.4 - Concepts and Applications* .Tata McGraw Hill**
This is a excellent book stressing application. A simple personnel database is used to highlight the data processing value of Unix. The chapters on Filters is very exhaustive. A good book for sustained self study.
8. **Vijay Mukhi: *Working with Unix*. BPB**
This book is written in an informal style but the chapters on Vi Editor and Device Management are well presented.

